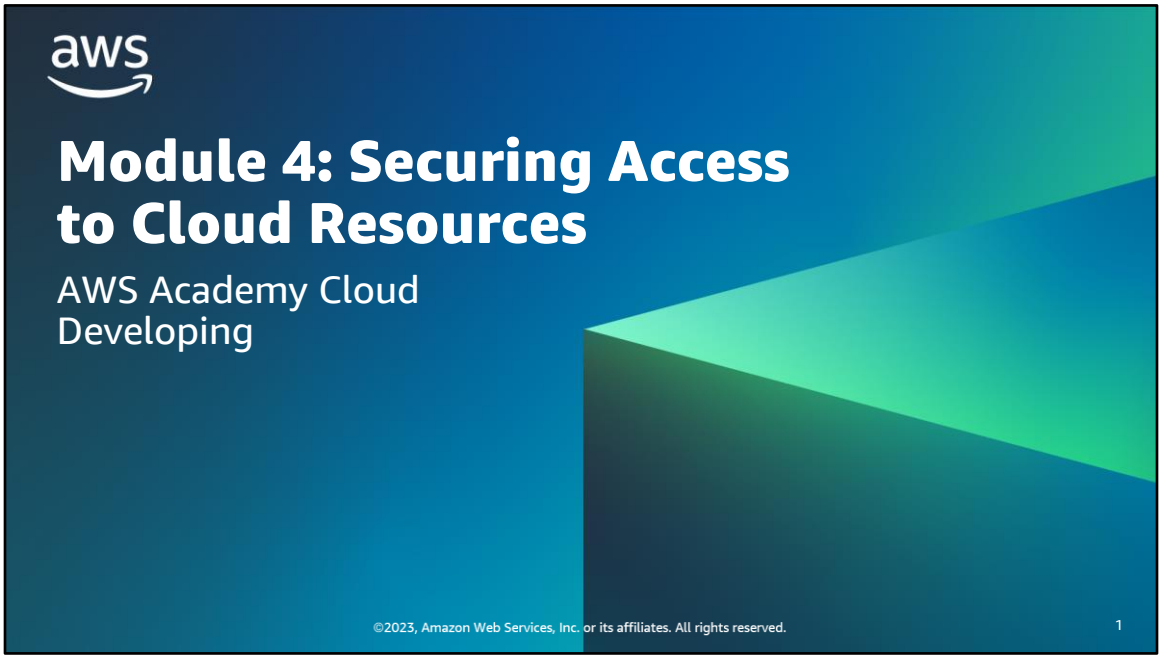
AWS Academy Cloud Developing
Module 04 Student Guide
Version 2.0.6

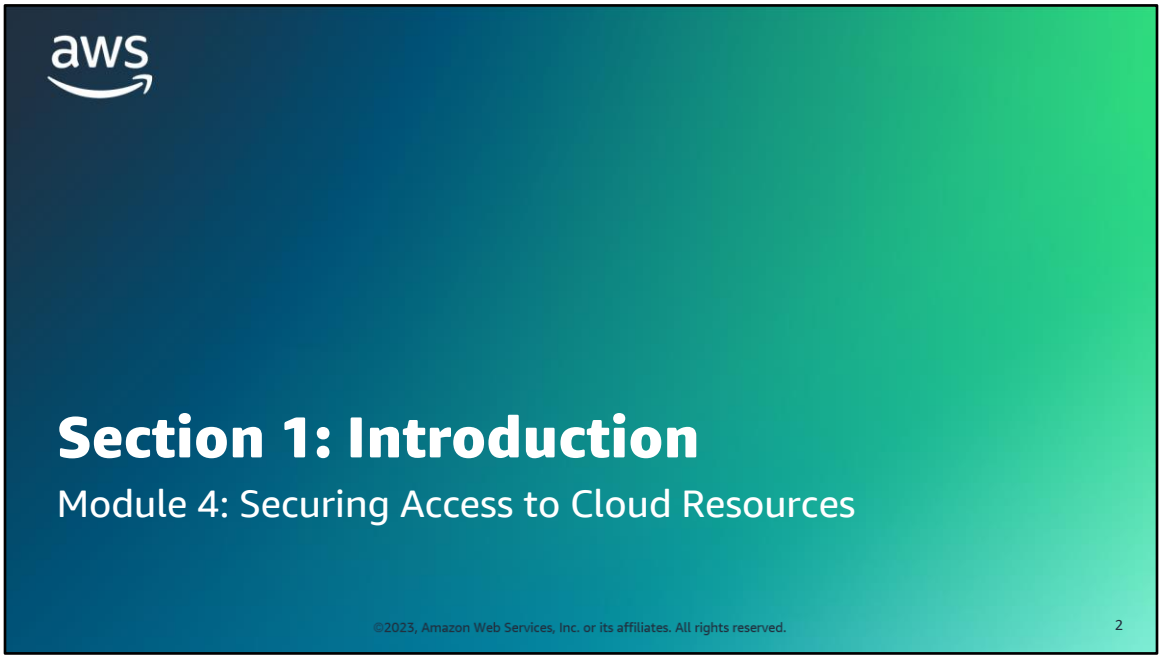
200-ACCDEV-20-EN-SG

Contents

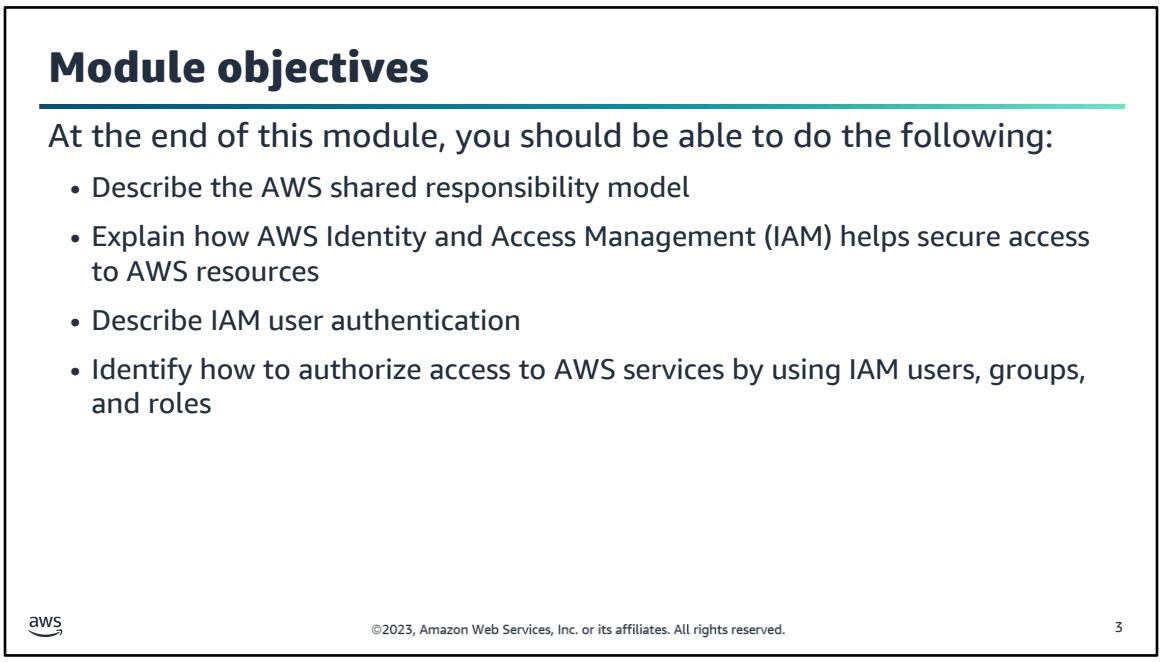
Module 4: Securing Access to Cloud Resources 4



Welcome to Module 4: Securing Access to Cloud Resources.

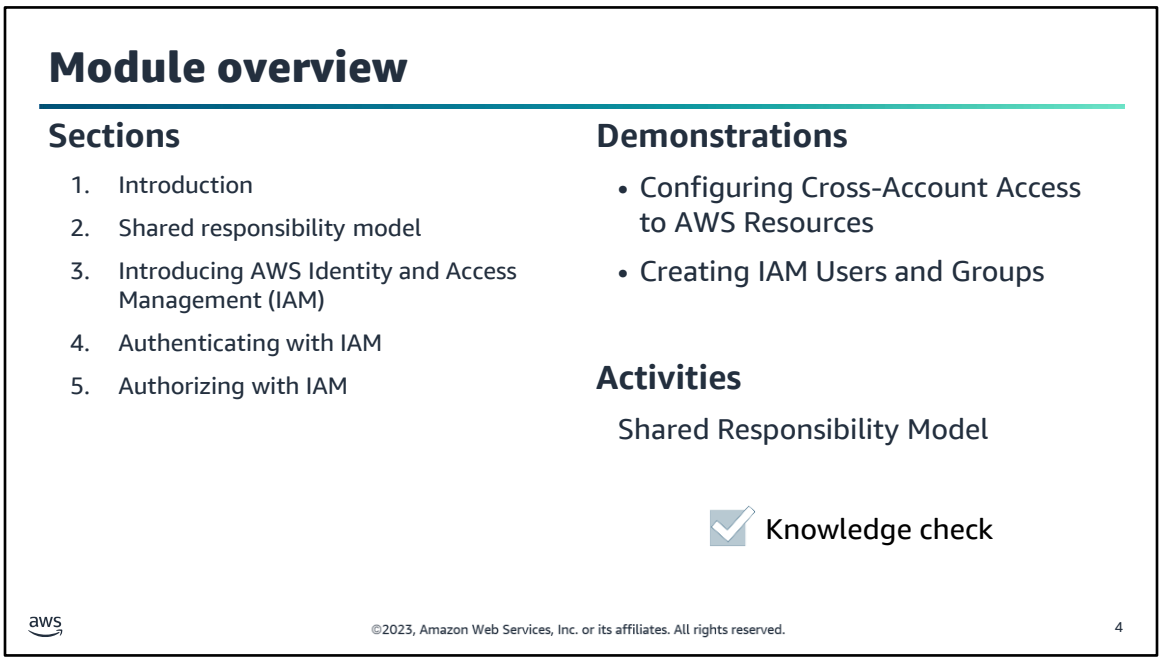


Section 1: Introduction.



At the end of this module, you should be able to do the following:

- Describe the AWS shared responsibility model
- Explain how AWS Identity and Access Management (IAM) helps secure access to AWS resources
- Describe IAM user authentication
- Identify how to authorize access to AWS services by using IAM users, groups, and roles



This module includes the following sections:

1. Introduction
2. Shared responsibility model
3. Introducing AWS Identity and Access Management (IAM)
4. Authenticating with IAM
5. Authorizing with IAM

This module also includes:

- An activity that is related to the AWS shared responsibility model
- A demonstration on how to create an IAM user and an IAM group
- A demonstration of how to configure cross-account access to AWS resources

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

Café business requirement

Frank and Martha liked the proof-of-concept website, and now other team members are going to help Sofía to develop the café's web application. Before development continues, she decides to define the level of access that users and systems should have across cloud resources.

aws

5

Frank and Martha liked the proof-of-concept website, and now other team members are going to help Sofía to develop the café's web application. Before development continues, she decides to define the level of access that users and systems should have across cloud resources.

With IAM, Sofía can define roles to give to other team members, based on what they will build, maintain, or access. For example, she might create a developer role and a database administrator role. Sofía will use IAM to clearly define what level of access each user should have to the AWS resources that the café website uses.

Section 2: Shared responsibility model.

Shared responsibility model principles

- **Security** and **compliance** – shared responsibility between AWS and the customer
- **AWS** responsibility:
  - Security **of** the cloud
- **Customer** responsibility:
  - Security **in** the cloud
- Objectives:
  - Reduce the customer's operational burden
  - Provide needed flexibility and control to the customer

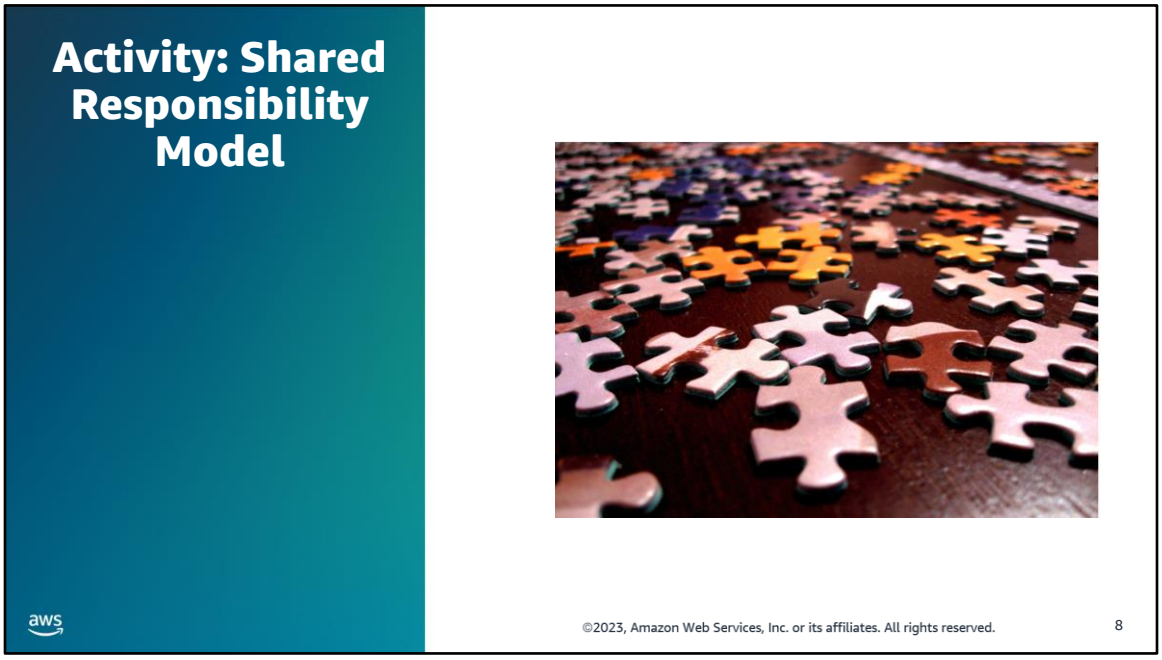
aws

7

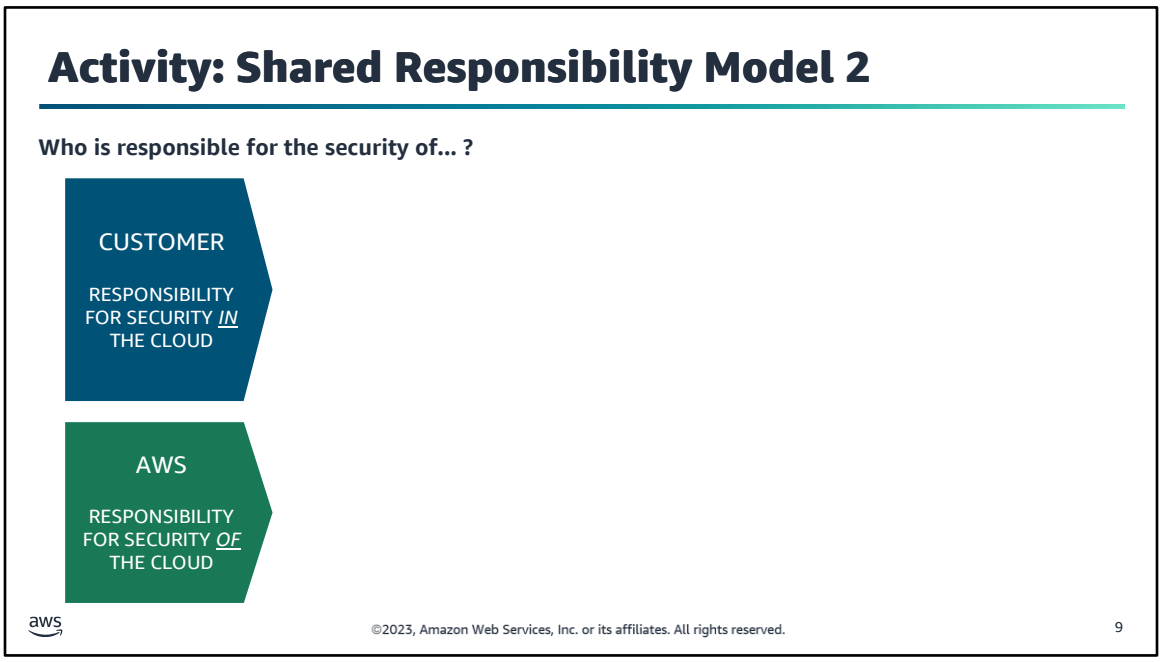
Ensuring security and compliance is a shared responsibility between AWS and the customer. It's important to differentiate who is responsible for what. Thus, it's helpful to summarize that AWS is responsible for security ***of*** the cloud, but the customer is responsible for security ***in*** the cloud.

The actual customer responsibility details are determined by the AWS Cloud services that a customer chooses to use. However, customers are generally responsible for managing their data. Customers are also responsible for using IAM and other security services or features to configure the appropriate implementation-specific security.

With the shared responsibility model, AWS provides security of the cloud's physical infrastructure, which the customer might have previously managed. As a result, the customer's operational burden is reduced. In addition, the nature of this shared responsibility also provides customers with flexibility and control. They need this flexibility and control to build custom applications and to make practical use of cloud computing resources.



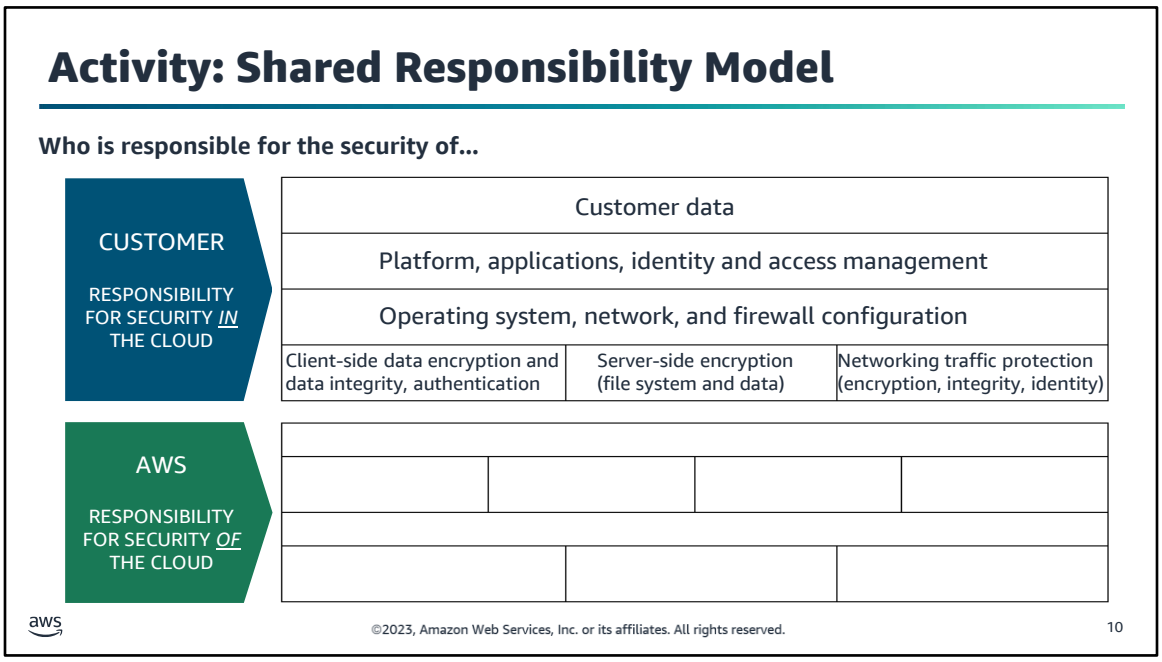
In this activity, the educator leads the class in a discussion of different areas of security concern. They classify each area as being the responsibility of AWS or the responsibility of the customer.



AWS operates, manages, and controls security *of* the cloud. This responsibility includes securing components, from the host operating system and virtualization layer down to the physical security of the facilities where the service operates. AWS is responsible for protecting the global infrastructure that runs all the services that are offered in the AWS Cloud. The global infrastructure includes AWS Regions, Availability Zones, and edge locations.

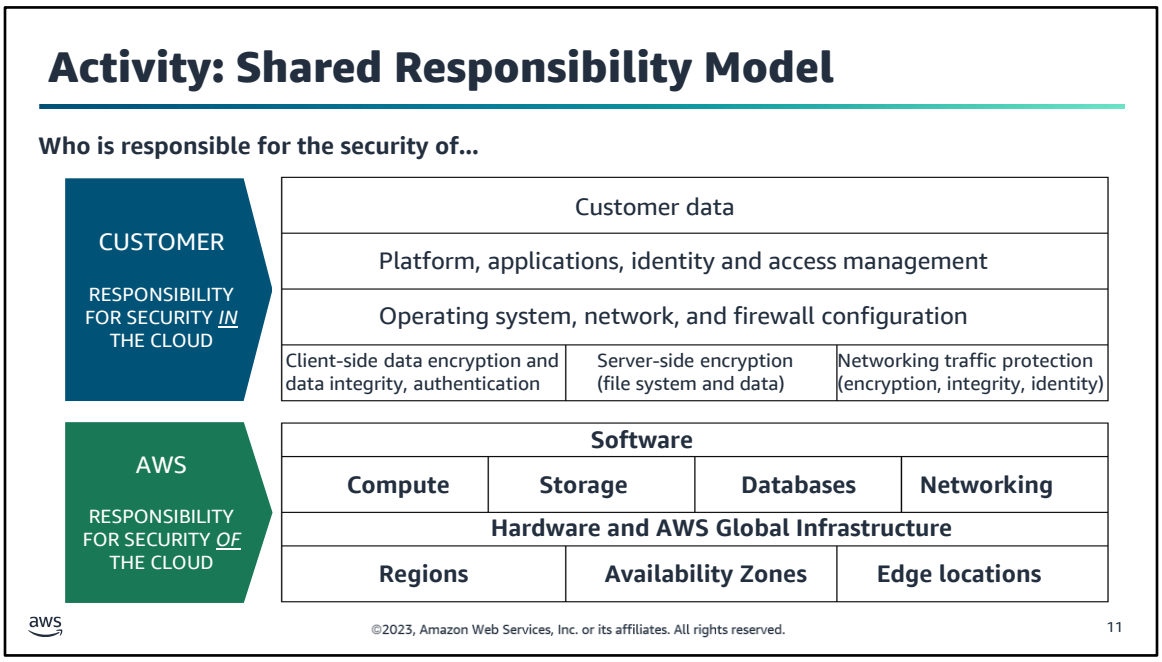
You assume responsibility and management *in* the cloud. This responsibility includes security of the guest operating system (including updates and security patches). It also includes other associated application software, and the configuration of the security group firewall that AWS provides. The **customer is responsible** for what is implemented by using AWS services and for the applications that are connected to AWS. The security steps that you must take depend on the services that you use and the complexity of your system. Customer responsibilities include selecting and securing operating systems that run on Amazon Elastic Compute Cloud (Amazon EC2) instances, and securing the applications that are launched on AWS resources. Customers must also select and handle security group configurations, firewall configurations, network configurations, and secure account management.

To reiterate, AWS secures the hardware, software, facilities, and networks that run all of AWS products and services. You are responsible for what you implement by using AWS products and services, and for the applications that you connect to AWS. The security steps that you must take depend on the services that you use and the complexity of your system.



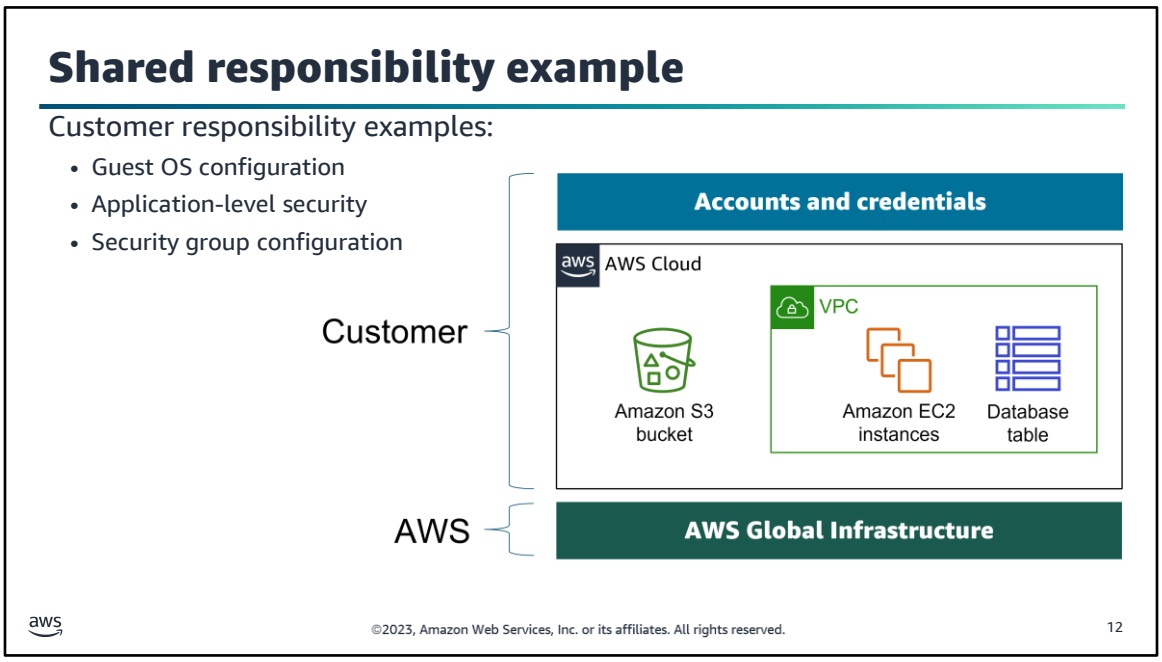
You assume responsibility and management *in* the cloud. This responsibility includes security of the guest operating system (including updates and security patches). It also includes other associated application software, and the configuration of the security group firewall that AWS provides. The **customer is responsible** for what is implemented by using AWS services and for the applications that are connected to AWS. The security steps that you must take depend on the services that you use and the complexity of your system. Customer responsibilities include selecting and securing operating systems that run on Amazon Elastic Compute Cloud (Amazon EC2) instances, and securing the applications that are launched on AWS resources. Customers must also select and handle security group configurations, firewall configurations, network configurations, and secure account management.

****For accessibility**: Customer is responsible for security in the cloud. Includes customer data, platform, applications, identity and access management. OS, network, and firewall configuration. Client-side data encryption and data integrity and authentication. Server-side encryption of file system and data. Networking traffic protection. **End description**.



To reiterate, AWS secures the hardware, software, facilities, and networks that run all of AWS products and services. You are responsible for what you implement by using AWS products and services, and for the applications that you connect to AWS. The security steps that you must take depend on the services that you use and the complexity of your system.

****For accessibility:** AWS is responsible for security of the cloud, including software, compute, storage, databases, networking, hardware and AWS Global infrastructure Regions, Availability Zones, and Edge locations. **End description**.



Consider an example where you work for a company, and you use Amazon Simple Storage Service (Amazon S3) to store some of your company's data. You also created an Amazon Elastic Compute Cloud (Amazon EC2) instance and an Amazon Relational Database Service (Amazon RDS) instance. These instances run a MySQL database that's deployed inside a virtual private cloud (VPC). The EC2 instance hosts a web server, and the web application that runs on it uses the database to store application data.

In this scenario, *AWS is responsible* for protecting the global infrastructure, which contains the physical servers that host the virtual machines and storage hardware. These virtual machines and storage hardware host your S3 bucket, EC2 instances, and database instance. AWS is responsible for the security of the physical networking infrastructure that ensures that these components can be accessed. AWS is also responsible for the security of the hypervisor layer that hosts the EC2 instances (the hypervisor is the host OS that runs the EC2 instances, which are virtual machines that run guest OSs).

You (the customer) are responsible for managing the guest OS that runs on the EC2 instances (including Microsoft Windows or Linux OS updates and security patches). You are also responsible for managing any application software or utilities that you install. Additionally, you are responsible for the configuration of the security groups that control network access to each EC2 instance and to the RDS database instance. You are also responsible for configuring security on the S3 bucket and the objects that you store in it. For example, you could use one or more of the security features that AWS provides, such as bucket policies, data encryption, and S3 block public access.

Section 3: Introducing AWS Identity and Access Management (IAM).

AWS Identity and Access Management (IAM)

- Securely controls individual and group access to your AWS resources
- Integrates with other AWS services
- Supports federated identity management
- Supports granular permissions
- Supports multi-factor authentication (MFA)

AWS Identity and Access Management (IAM)

 14

AWS Identity and Access Management is also known as IAM. It's a service that you can use to configure fine-grained access control to AWS resources. With IAM, you can follow security best practices by granting unique security credentials to roles, users, and groups. These credentials specify which AWS service application programming interfaces (APIs) and resources these users can access. Users have no access to AWS resources until permissions are explicitly granted.

IAM is *integrated into most AWS services*. You can define access controls from one place in the AWS Management Console, and they will take effect throughout your AWS environment.

IAM can be used to grant employees and applications access to the AWS Management Console and to AWS service APIs by using existing identity systems. AWS supports *federation from corporate systems* like Microsoft Active Directory and standards-based identity providers.

IAM also supports *multi-factor authentication (MFA).* If MFA is enabled and an IAM user attempts to log in, the user is prompted for an authentication code. The authentication code is delivered to an AWS MFA device, which might be a hardware MFA device. It can also be a virtual MFA device that the user accesses through an application that runs on the user's smartphone, such as Google Authenticator.

Security fundamentals

Authentication

- **Who** is requesting access to the AWS account and the resources in it?
- It's important to establish the identity of the requester through credentials.
- The requester could be a person or an application – IAM calls them principals.

Authorization

- Assuming that the requester has been authenticated, **what** should they be allowed to do?
- IAM checks for policies that are relevant to the request to determine whether to allow or deny the request.

aws

15

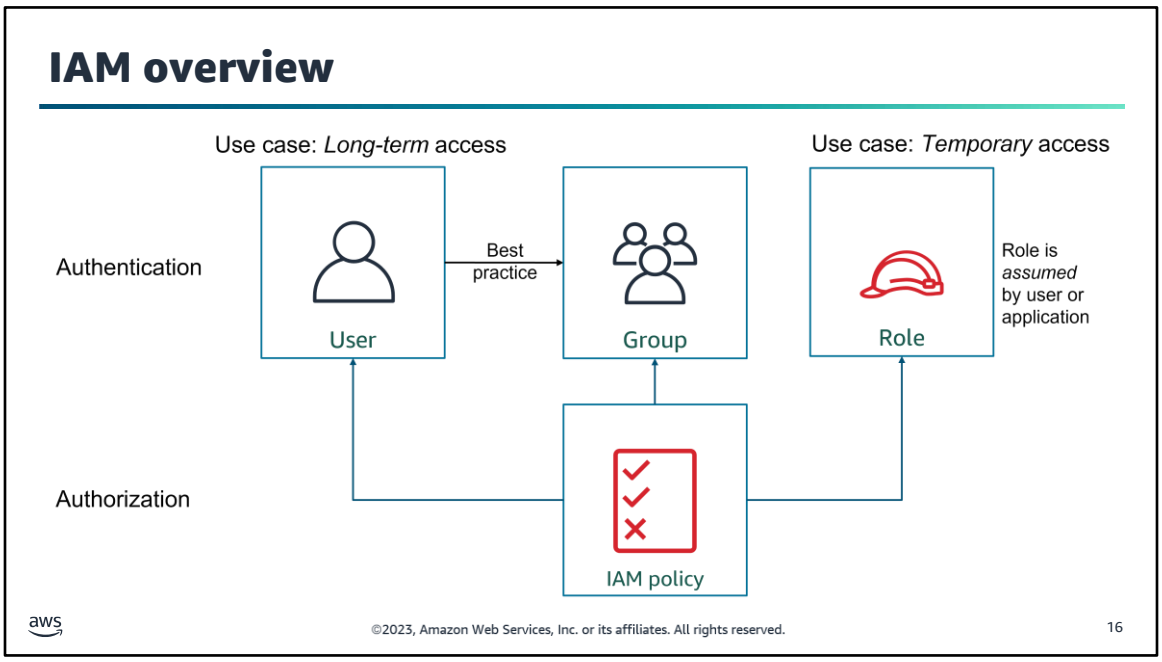
With IAM, you can control *who* can use your AWS resources (this component is the identity, or authentication, component). You can also control *what* resources they can use and in what ways (this component is the access management, or authorization, component).

To understand the fundamental aspects of authentication and authorization, consider a banking analogy. Think of a bank that allows their customers (users) to access their accounts online. Suppose that you are a bank customer. You have money in a checking account at that bank, and you want to pay a bill online. Should any random person be able to log in to your bank account and pay their bills with the money in your account? No, a random person shouldn't be able to use the money in your account.

Before the bank allows you to access your checking account, it must ensure that the person who is accessing the account is you. The bank wants to ***authenticate*** that you are who you claim to be. They typically accomplish this authentication by requiring you to enter your user name, and a password that is supposedly known only to you. For extra security, they might have configured two-factor authentication. Thus, in addition to typing in a password, you also must receive a code on your phone and enter that code.

Now, suppose that you are successfully logged in to the bank website (authenticated). Can you pay your bill by using the money in some *other customer's* account? No, you shouldn't be able to use another customer's money to pay your bill. You don't have the ***authorization*** to access accounts that don't belong to you. However, you are authorized to access the money in your account, and you are authorized to pay bills by using your money.

How does this analogy relate to IAM? Similar to how a bank must secure access to resources (your money), you—as an AWS account owner—must secure access to your AWS account. You want whatever data that you store in your account to be secure so that others can't access it. Likewise, you want to secure the application logic of anything that you build on AWS, so that others can't modify it. IAM provides many features that can help you to accomplish these security objectives.

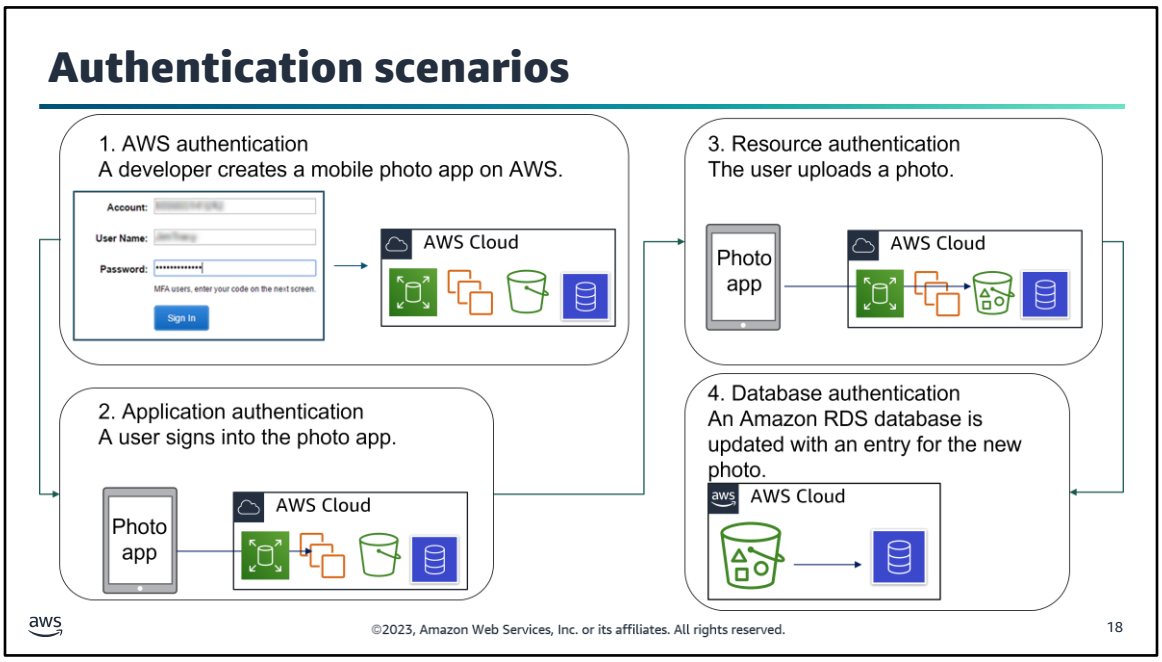


You can use IAM to control access to your AWS resources. You achieve this access control by creating users, groups, and roles. You can also enforce access control by attaching policies.

- An *IAM user* is an entity that you create in AWS to represent a person or application that interacts with AWS account services and resources. A user is given a permanent set of credentials, which stay with the user until a forced rotation occurs.
- An *IAM group* is a collection of IAM users. You can use groups to grant the same set of permissions to multiple users.
- An *IAM role* is similar to a user. It's an AWS identity that you can attach permission policies to. These policies determine what the identity can and can't do in AWS. However, a role doesn't have long-term credentials (such as a password or access keys) that are associated with it. Instead, when a person or application assumes a role, they are provided with temporary security credentials for the role session.
- An *IAM policy* is a document that explicitly lists permissions. The policy can be attached to an IAM user, to an IAM group, to an IAM role, or any combination of these resources.

To configure long-term access, a best practice is to attach IAM policies to IAM groups, and then assign IAM users to these IAM groups. An IAM user who is a member of an IAM group inherits the permissions that are attached to that group. You can also attach IAM policies directly to an IAM user to further customize the access that's granted through the group.

Section 4: Authenticating with IAM.



When you develop applications, you might have different times when authentication is required for various people or applications.

For example, suppose that you are developing a mobile photo app. Users can use the app take pictures with their mobile device and then upload the pictures to AWS.

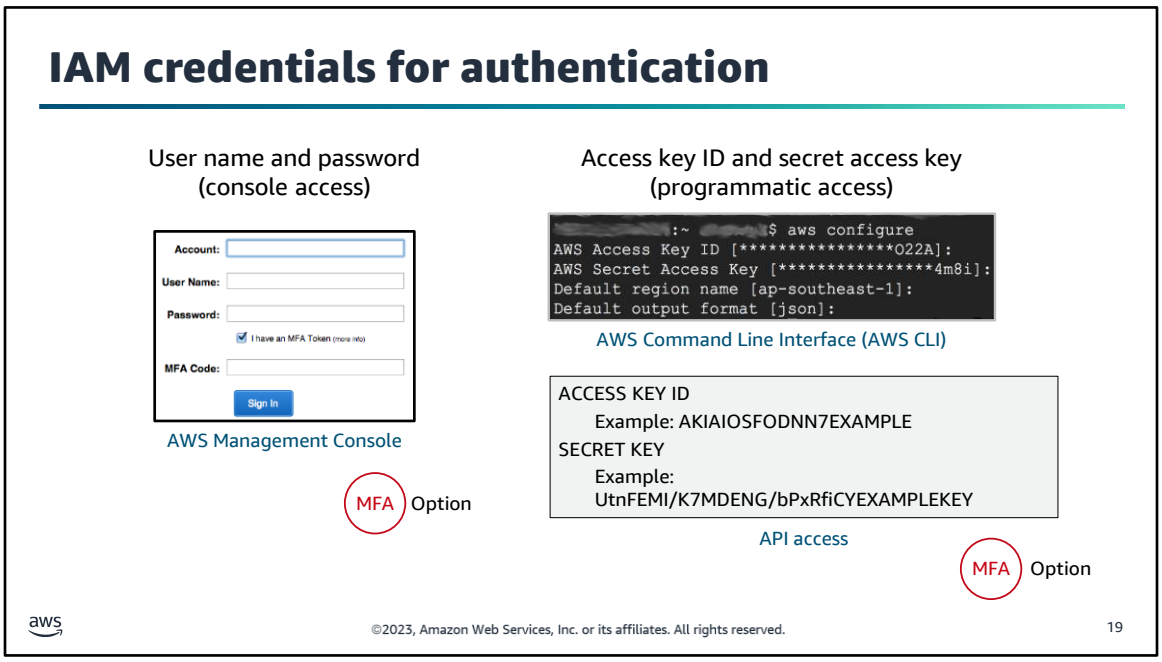
As **the developer**, you must use your AWS credentials to authenticate to the AWS account, and confirm that you are who you say you. In this way, you can access the AWS account and build the photo app (*AWS account authentication*).

Likewise, **an app user** must sign in to the photo app, which requires that they authenticate with your app (*application authentication*).

Additionally, when the user uses your photo app to take a picture and upload it, **the photo app** must authenticate to the AWS account. It must authenticate before the account allows the application to access a service—such as Amazon S3—where the photos will be stored (*resource authentication*).

Finally, the photo that's uploaded to the S3 bucket can trigger a backend job to update an Amazon RDS database. The database will be updated with an entry for the uploaded photo. This job requires *database authentication*.

****For accessibility**: 1. AWS authentication is required when a developer creates a mobile photo app on AWS. 2. Application authentication is needed when a user signs into the photo app. 3. Resource authentication is required when a user of the app uploads a photo, which goes into a Bucket. 4. Database authentication must happen for the Amazon RDS database to be updated with the new photo. **End description.**



You must authenticate to any supported services from the AWS Management Console, AWS Command Line Interface (AWS CLI), and AWS software development kits (SDKs). To do so, you must provide AWS credentials.

Two primary types of credentials are used for authentication. Which credential type you use depends on how you access AWS.

- To authenticate from the console, you must sign in with your user name and password.
- To authenticate programmatically through the AWS CLI, SDKs, and application programming interfaces (APIs), you must provide an AWS access key. The AWS access key is the combination of an access key ID and a secret access key.

You might also be required to provide additional security information. For example, AWS recommends that you use multi-factor authentication (MFA) to increase the security of your account.

For more information, see “AWS Security Credentials” at https://docs.aws.amazon.com/general/latest/gr/aws-security-credentials.html.

Multi-factor authentication (MFA)

Adds an extra layer of protection on top of your user name and password

- Users are prompted for an authentication code
- It can be hardware-based or a virtual device

Enable MFA for:

- AWS Management Console users
- AWS API users (requires temporary security credentials)

 20

Multi-factor authentication (MFA) is a best practice that adds an extra layer of protection on top of your user name and password. MFA requires two or more factors to achieve authentication. Factors include something you know, such as a password; something you have, such as a cryptographic token; or something you are, such as your fingerprint.

- When users sign in to the AWS Management Console, they are prompted for their user name and password (the first factor—what they know). They are then prompted for an authentication response from their AWS MFA device (the second factor—what they have). Examples of MFA devices include YubiKeys and Gemalto devices.
- You can also enforce MFA by adding MFA restrictions to your IAM policies. You might need access to APIs and resources that are protected in this way. If so, you can request temporary security credentials and pass optional MFA parameters in their AWS Security Token Service (AWS STS) API requests. MFA-validated temporary security credentials can be used to call MFA-protected APIs and resources.

AWS Security Token Service (AWS STS) is the service that issues temporary security credentials. This service will be covered in more detail later in this course.

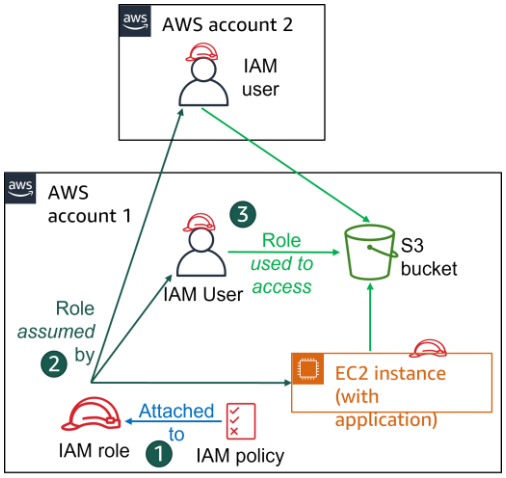
IAM roles provide temporary credentials

IAM role characteristics

- Provides temporary security credentials
- Is not uniquely associated with one person
- Is assumable by a person, application, or service
- Is often used to delegate access

Common use cases

- Application that runs on Amazon Elastic Compute Cloud (Amazon EC2)
- Cross-account access for an IAM user
- Mobile applications



21

An IAM *role* enables you to define a set of permissions to access the resources that a user or service needs. However, the permissions are not attached to an IAM user or group. Instead, the permissions are attached to a role, and the user or the service *assumes* the role.

When a user assumes a role, the user's prior permissions are temporarily forgotten. AWS returns temporary security credentials that the user or application can then use to make programmatic requests to AWS.

When you use IAM roles, you don’t need to grant long-term security credentials to each entity that requires access to a resource.

For a service like Amazon EC2, applications or AWS services can programmatically assume a role at runtime. The principal that assumes the role might be an IAM user, group, or role from another AWS account, including accounts that you do not own.

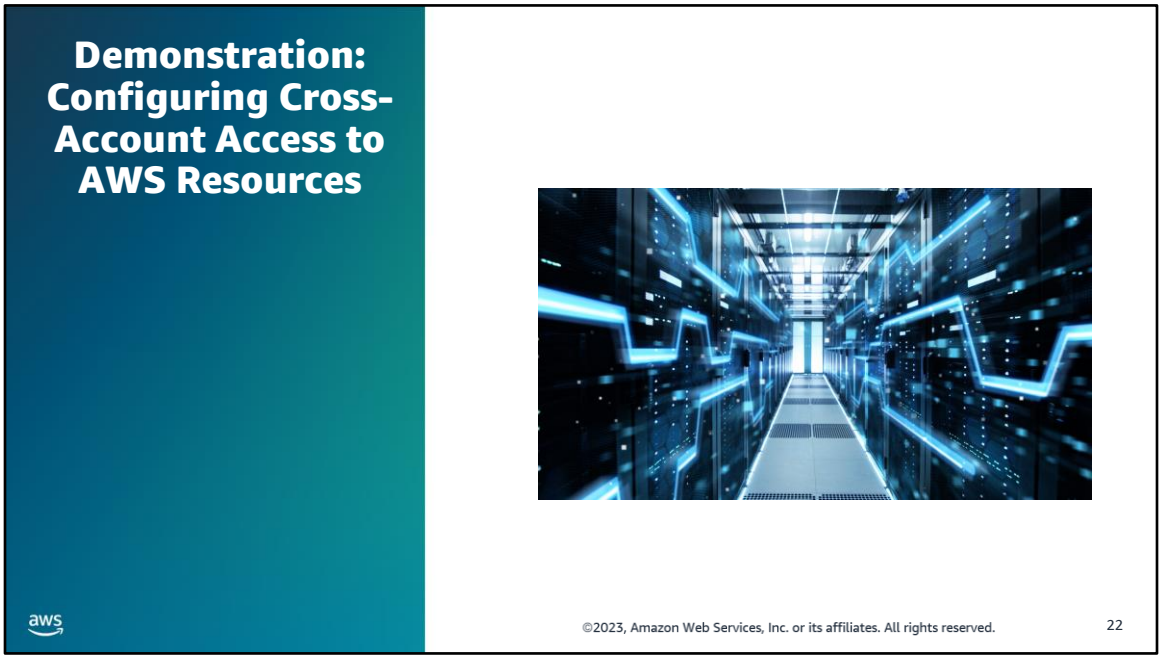
By creating a role for *external account access*, you don’t need to manage user names and passwords for third parties. If you no longer want someone or some system to have access, you can modify or delete the role. Thus, you don’t need to create and manage accounts for people outside your organization.

At times, you might need to provide trusted users with temporary security credentials to your AWS resources. In this case, it’s a best practice to create an IAM role with temporary credentials instead of creating an IAM user.

The following examples illustrate situations when you might need to use temporary security credentials:

- An IAM user in one AWS account might need temporary access to resources in another AWS account.
- User identities that were authenticated by a system outside AWS might need to perform AWS tasks and access your AWS resources. Examples of such external systems might be an enterprise identity provider or web identity provider. This process is referred to as identity federation.
- A mobile app might need access to your AWS resources. You don’t want to store AWS credentials with the application, where they can be difficult to rotate and where users can potentially extract them.
- You can provide temporary security credentials to applications that run on EC2 instances and other AWS compute services. In this way, you don’t need to store long-term credentials locally.

***For accessibility:** An IAM policy is attached to an IAM role in AWS account 1. The role is assumed by an IAM user in the same account and an IAM user in AWS account 2. Both users can use the role to access in S3 bucket. An EC2 instance also assumes the same role and the app running on the instance can also access the S3 bucket. **End description**.



There is a video demonstration available for this topic. You can find this video within the module 4 section of the course with the title **Demo Configuring Cross-Account Access to AWS Resources**.

If you are unable to locate this video demonstration, please reach out to your educator for assistance.

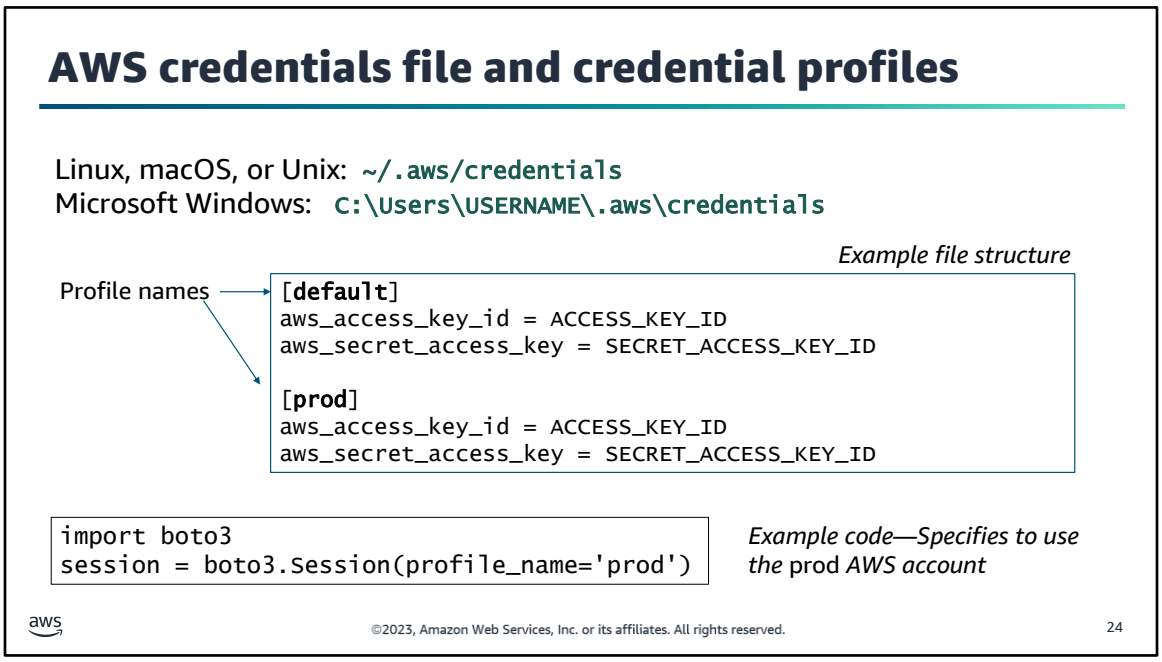
Providing access to AWS resources through your IDE

- Use AWS toolkits to integrate access to AWS resources.
- Use AWS managed temporary credentials for IDEs installed on an EC2 instance.

 23

AWS provides toolkits for many popular IDEs to help integrate access to AWS resources into your IDE environment. Follow the documentation for your IDE and its toolkit for the details of how to provide your IAM credentials through your IDE. See the Tools to Build on AWS page, IDE toolkits section at: https://aws.amazon.com/developer/tools/.

If you are running your IDE remotely on an EC2 instance, use temporary AWS access credentials. These credentials are called *AWS managed temporary credentials*. AWS managed temporary credentials manage AWS credentials in an Amazon EC2 environment on your behalf, and they also follow AWS security best practices. This method avoids distributing or embedding long-term credentials with an application or storing your permanent AWS credentials within the environment, which is less secure.

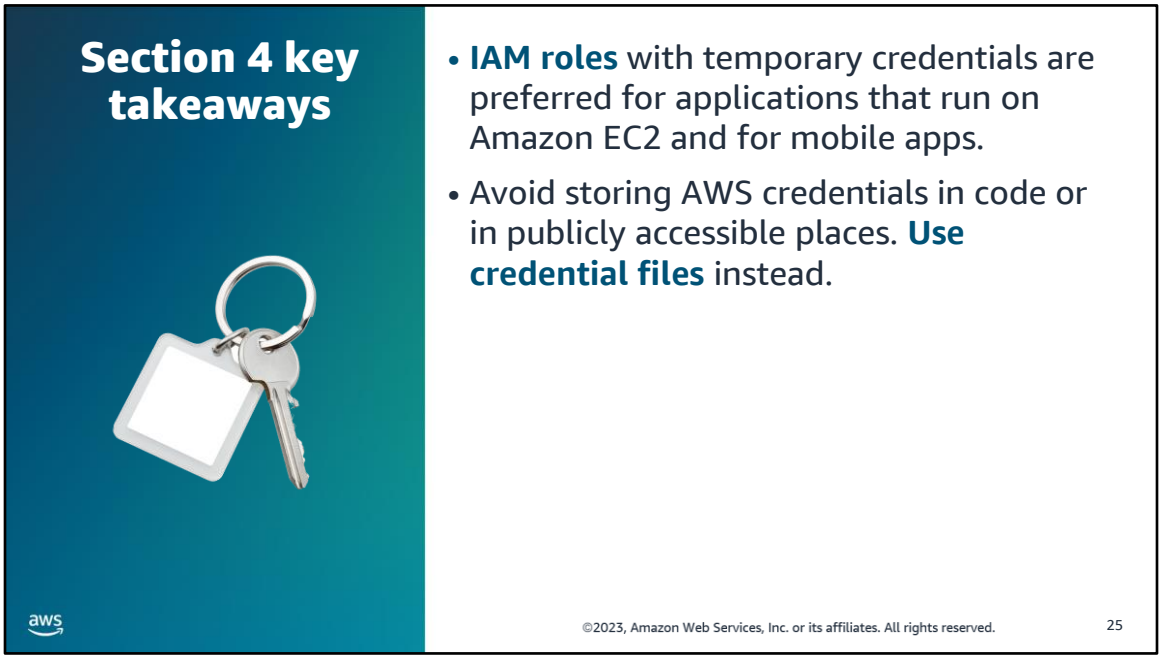


After the AWS CLI is installed, you can use it to interact with one or more AWS accounts. To set your AWS credentials so that the AWS CLI client can use them, run the **aws configure** command. The AWS CLI stores the credentials that you set in your home directory, in a local file named **credentials** that's in a folder named **.aws**. The default location for your credentials file is **~/.aws/credentials** on Linux, macOS, or Unix. On Microsoft Windows, the default location is **C:\Users\USERNAME\.aws\credentials**.

Using an AWS credentials file offers a few benefits:

- Your projects' credentials are stored outside of your projects, so it's unlikely that you will accidentally commit them into version control.
- You can define and name multiple sets of credentials (called *profiles*) in one place. In this example credentials file, you see AWS access key credentials for two *profiles*—one for *default* and one for *prod* (or production).
- You can reuse the same credentials between projects.
- You can reference your credentials when you are using other supported AWS SDKs and tools (such as in the example code).

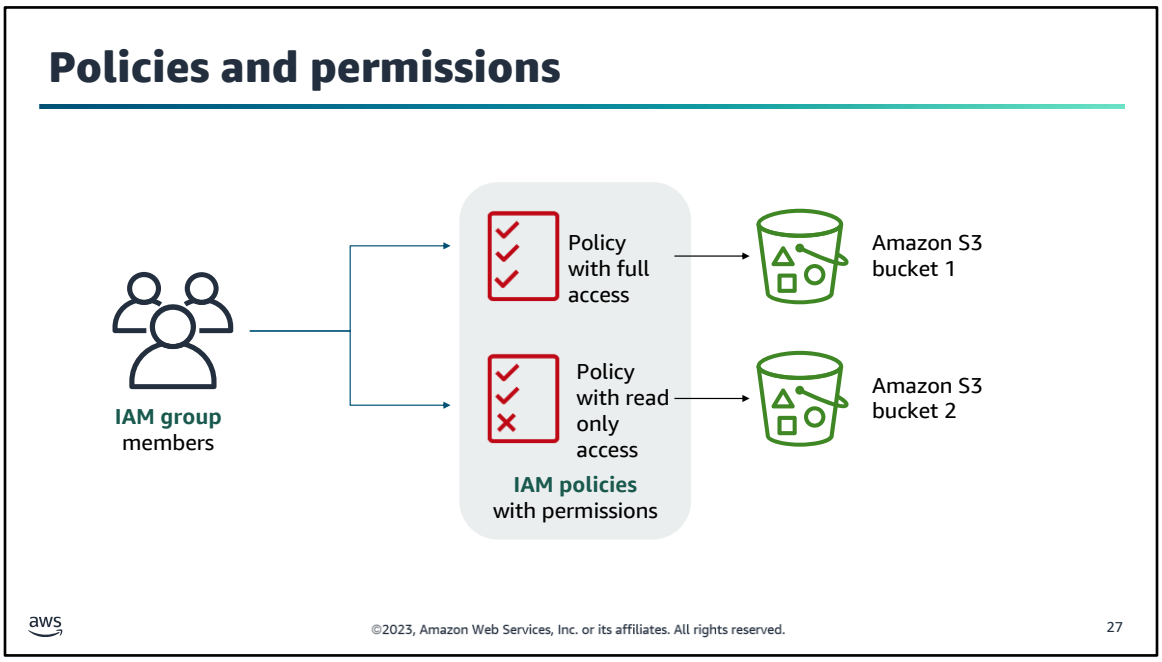
When you use the AWS CLI, you can also specify which profile you want to use. You can use the AWS CLI to switch between profiles and access keys. You can reference profiles from an SDK configuration file, or by using the profile option when you are instantiating a client.



The following are the key takeaways from this section of the module:

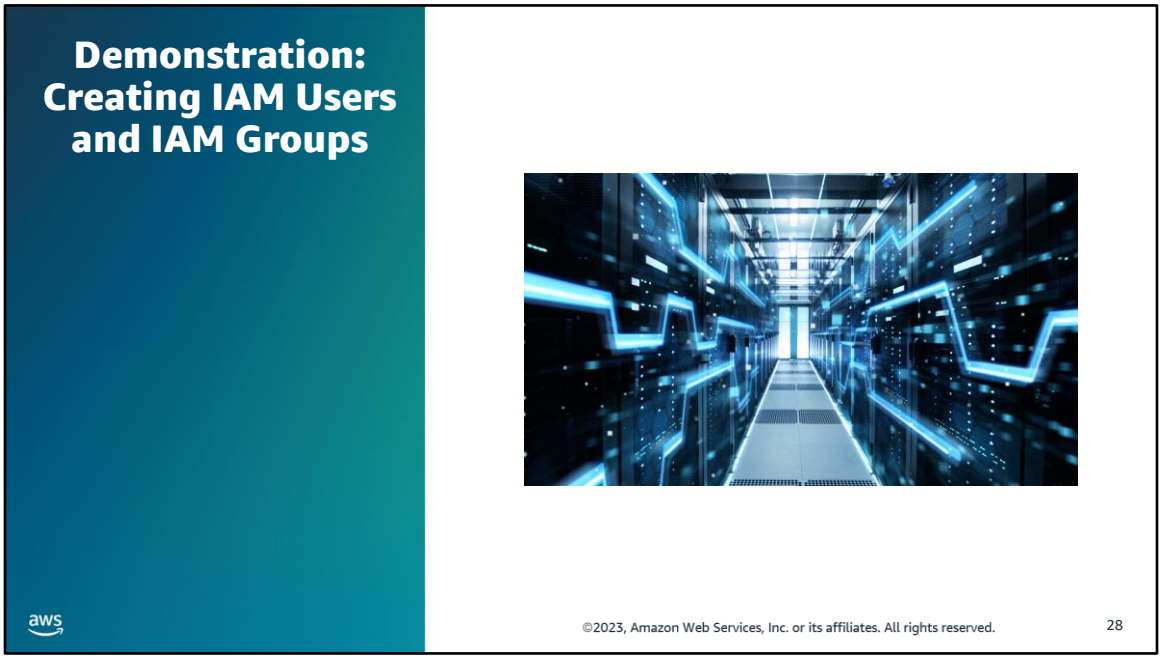
- IAM roles with temporary credentials are preferred for applications that run on Amazon EC2 and for mobile apps.
- Avoid storing AWS credentials in code or in publicly accessible places. Use credential files instead.

Section 5, Part1: Authorizing Access with IAM.



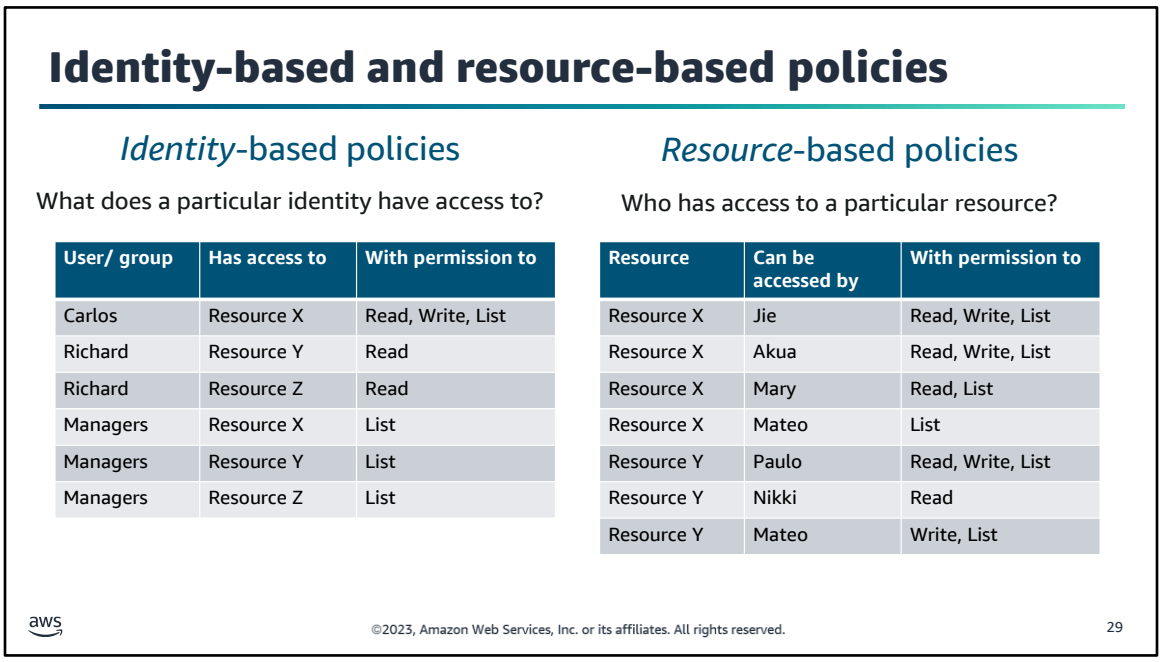
You can use IAM to control access to your AWS resources by using IAM policies that grant or deny permissions. For example, the users in the example IAM group can read, write, and delete objects in bucket 1. However, they can only read the objects in bucket 2.

By default, an authenticated IAM user, group, or role can't access anything in your account until you grant them *permissions*. You grant permissions by creating a *policy*. An IAM policy is a JavaScript Object Notation (JSON) document. It defines the effect, actions, resources, and optional conditions under which an entity can invoke API operations to the AWS account. By default, any actions or resources that are not explicitly allowed are denied.



There is a video demonstration available for this topic. You can find this video within the module 4 section of the course with the title **Demo Creating IAM Users and IAM Groups**.

If you are unable to locate this video demonstration, please reach out to your educator for assistance.



Two types of IAM policies can grant permissions: identity-based policies and resource-based policies. The difference between the two types of policies is a result of where they are applied.

- *Identity-based policies* are attached to an IAM user, group, or role. They indicate what that identity can do. For example, you could grant a user the ability to access a DynamoDB table.
- *Resource-based policies* are attached to a resource. They indicate what a specified user (or group of users) is permitted to do with it. For example, you can use resource-based policies to grant access to an S3 bucket, or to grant cross-account access between two trusted AWS accounts.

For more information, see “Identity-Based Policies and Resource-Based Policies” at https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_identity-vs-resource.html.

Example of identity-based policy

```
{
  "Version": "2018-10-17",
  "Statement": {
    "Effect": "Allow",
    "Action": [
      "iam:*LoginProfile",
      "iam:*AccessKey*"
      "iam:*SSHPublicKey*"
    ],
    "Resource": "arn:aws:iam::account-
id-without-hyphens:user/${aws:username}"
  }
}
```

The actions allowed

The AWS resources the action can be performed on

aws

30

After an identity-based policy is applied to a user, group, or role, it allows or denies the entity the ability to perform specified actions.

In the example shown, if the entity is a user, the policy allows the user to:

- Create, delete, get, or update their own password by using IAM `LoginProfile` actions.
- Create, delete, list, or update their own access key by using `IAM AccessKey` actions.
- Create, delete, get, list, or update their own SSH keys by using IAM `SSHPublicKey` actions.

The actions in the policy include wildcards, which are indicated with an asterisk (*). This policy is a convenient way to include a set of related actions. Notice that the policy is getting the AWS user name dynamically, as defined in the `"Resource“` key.

****For accessibility**: Code block shows actions allowed and AWS resources the action can be performed on. Allowed actions are Login Profile, Access Key, and SSH Public Key. **End description.**

Example cross-account, resource-based policy

```
{
  "Version": "2018-10-17",
  "Statement": {
    "Sid": "AccountBAccess1",
    "Principal": {"AWS": "111122223333"},
    "Effect": "Allow",
    "Action": "s3:*",
    "Resource": [
      "arn:aws:s3:::DOC-EXAMPLE-BUCKET",
      "arn:aws:s3:::DOC-EXAMPLE-BUCKET/*"
    ]
  }
}
```

Who can make the request (Account B)

Actions allowed

Resources shared by Account A

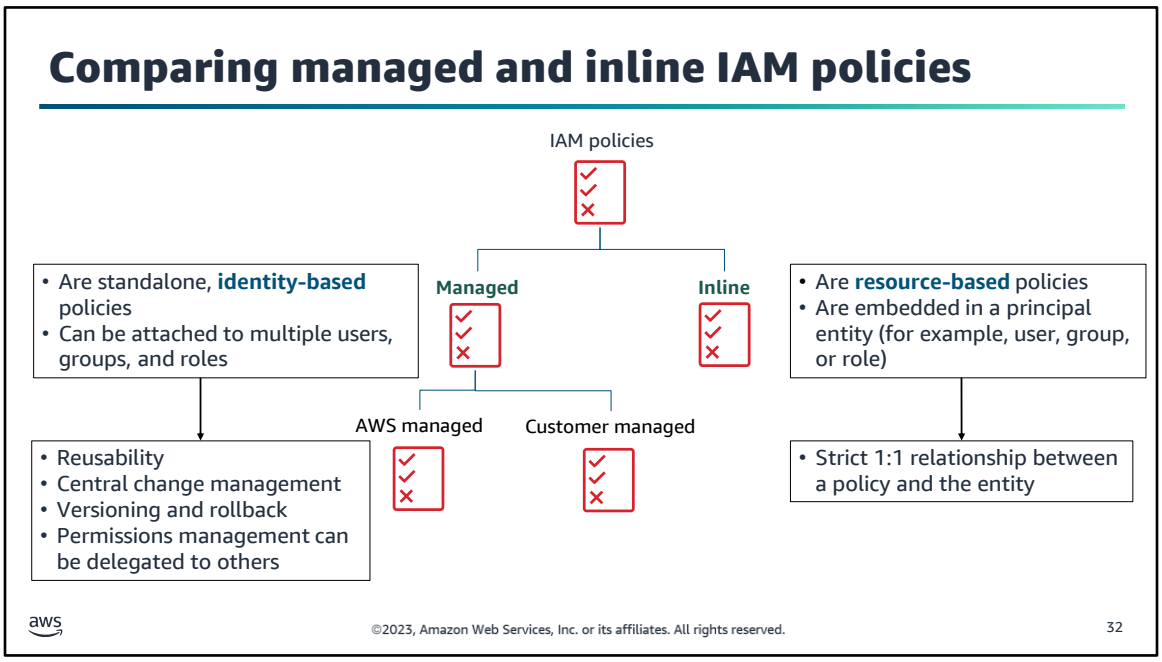
aws

31

This example shows a resource-based policy that allows some cross-account access.

In this scenario, **Account A** created a resource-based policy. The policy grants **Account B** access to perform *any* Amazon S3 API operation—which is indicated by the asterisk (*)—on Account A's S3 bucket (named *DOC-EXAMPLE-BUCKET)*.

This S3 bucket policy doesn’t specify any IAM users, groups, or roles. Instead, it specifies Account B (AWS account number *111122223333*). Account B should create an IAM user policy to allow a user in Account B to access Account A's bucket.



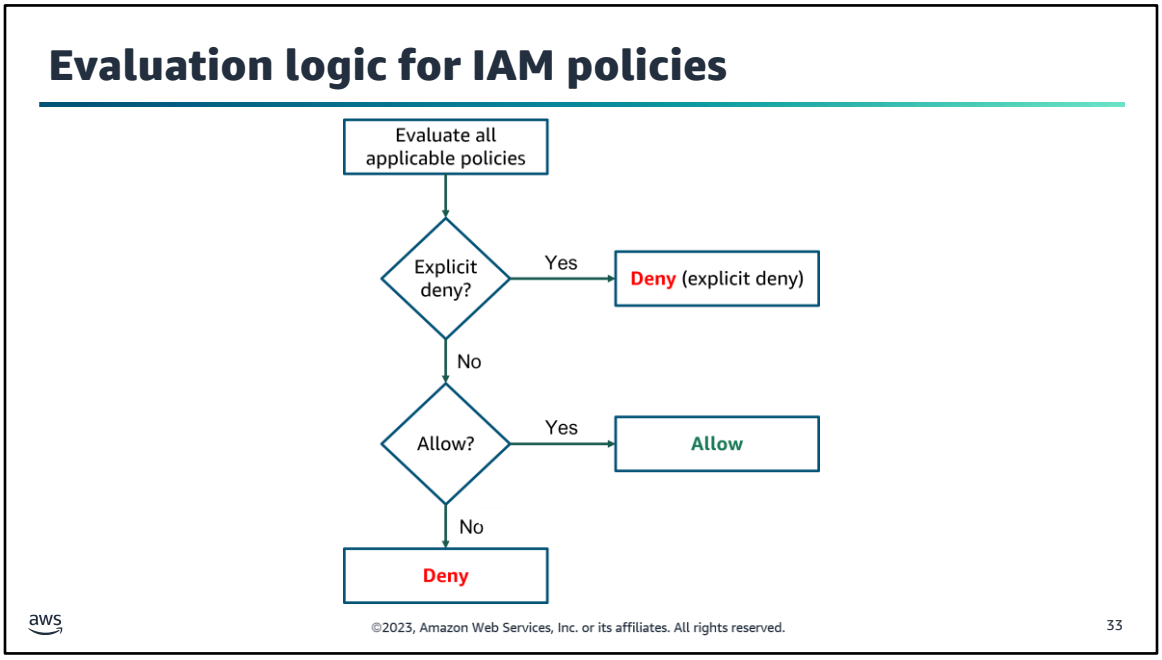
IAM policies can be managed or inline.

Managed policies are standalone, identity-based policies that you can attach to multiple users, groups, and roles.

- *AWS-managed policies* are created and managed by AWS.
- *Customer-managed policies* are created and managed by you.
- Managed policies provide several features, including reusability, central change management, versioning and rollback, and the ability to delegate permissions management to other users.

Inline policies are embedded in a principal entity such as a user, group, or role. That is, the policy is an inherent part of the entity.

- You can use the same policy for multiple entities, but those entities do not share the policy. Instead, each entity has its own copy of the policy, which makes it impossible to centrally manage inline policies.
- Inline policies are useful when you want to maintain a strict one-to-one relationship between a policy and the principal entity to which it is applied.



This diagram shows the logic that AWS uses when evaluating IAM policies. AWS evaluates all applicable policies and goes through this evaluation logic.

- By default, all requests are denied.
- An explicit allow overrides the default deny.
- An explicit deny overrides any explicit allow.

The order that the policies are evaluated in has no effect on the outcome of the evaluation. All policies are evaluated, and the result is always that the request is either allowed or denied. Suppose that a conflict occurs (one policy allows an action and another policy denies an action). Then, the most restrictive policy (that is, the policy that denies the action) is applied.

Example IAM policy: Allow statement

Box highlights the Effect Allow, Action, and Resources associated with the Allow statement.

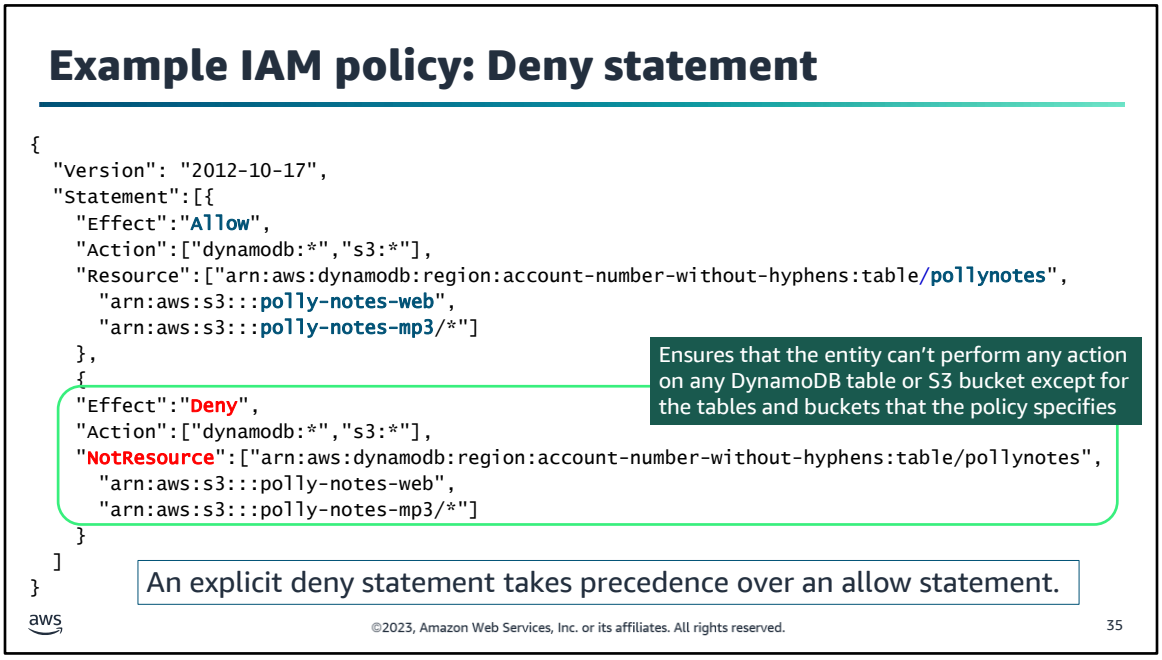
```
{
  "Version": "2012-10-17",
  "Statement":[{
    "Effect":"Allow",
    "Action":["dynamodb:*","s3:*"],
    "Resource":["arn:aws:dynamodb:region:<account-number-without-hyphens>:table/pollynotes",
      "arn:aws:s3:::polly-notes-web",
      "arn:aws:s3:::polly-notes-mp3/*"]
    },
    {
    "Effect":"Deny",
    "Action":["dynamodb:*","s3:*"],
    "NotResource":["arn:aws:dynamodb:region:<account-number-without-
hyphens>:table/pollynotes",
      "arn:aws:s3:::polly-notes-web",
      "arn:aws:s3:::polly-notes-mp3/*"]
    }
  ]
}
```

 34

To understand how IAM policy logic works, consider this example. The top half of this policy gives users access to only the following resources:

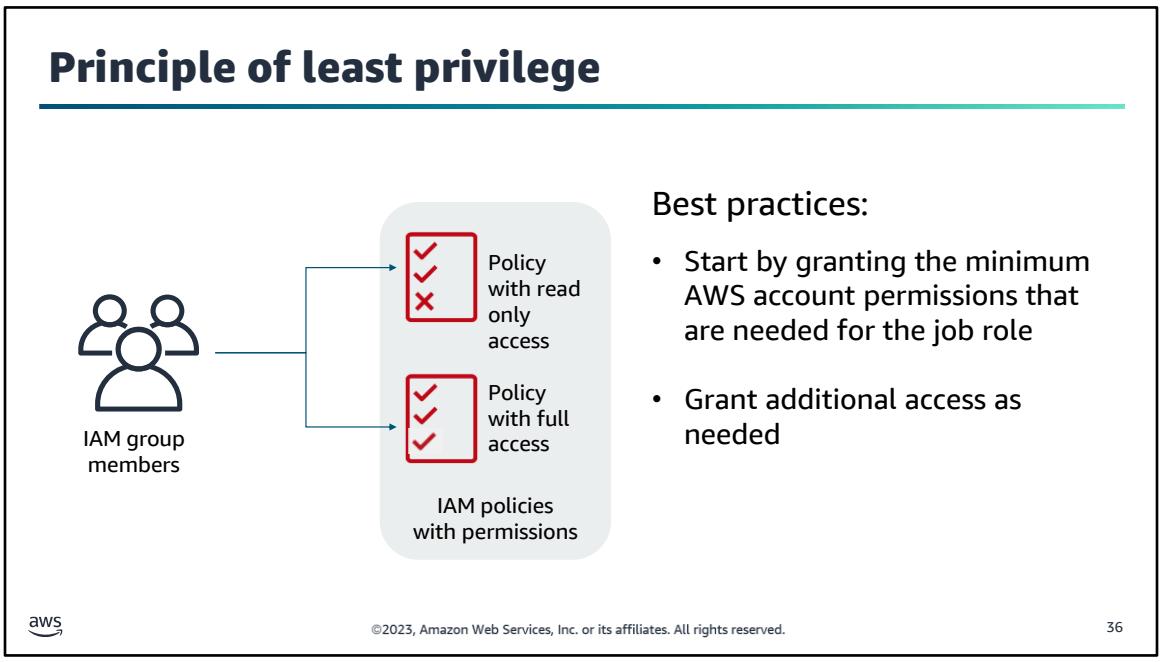
- The DynamoDB table *pollynotes*
- The S3 bucket *polly-notes-web*
- The S3 bucket *polly-notes-mp3* and all the objects that it contains

The second statement in the policy will be discussed on the next slide.



The bottom half of this policy is an explicit deny. Here, the explicit deny ensures that the entity can't perform any action on any DynamoDB table or S3 bucket. The only exceptions are the tables or buckets that are specified in the policy statement. Even if permissions are granted in another policy, the explicit deny overrides these permissions. An explicit deny statement takes precedence over an allow statement.

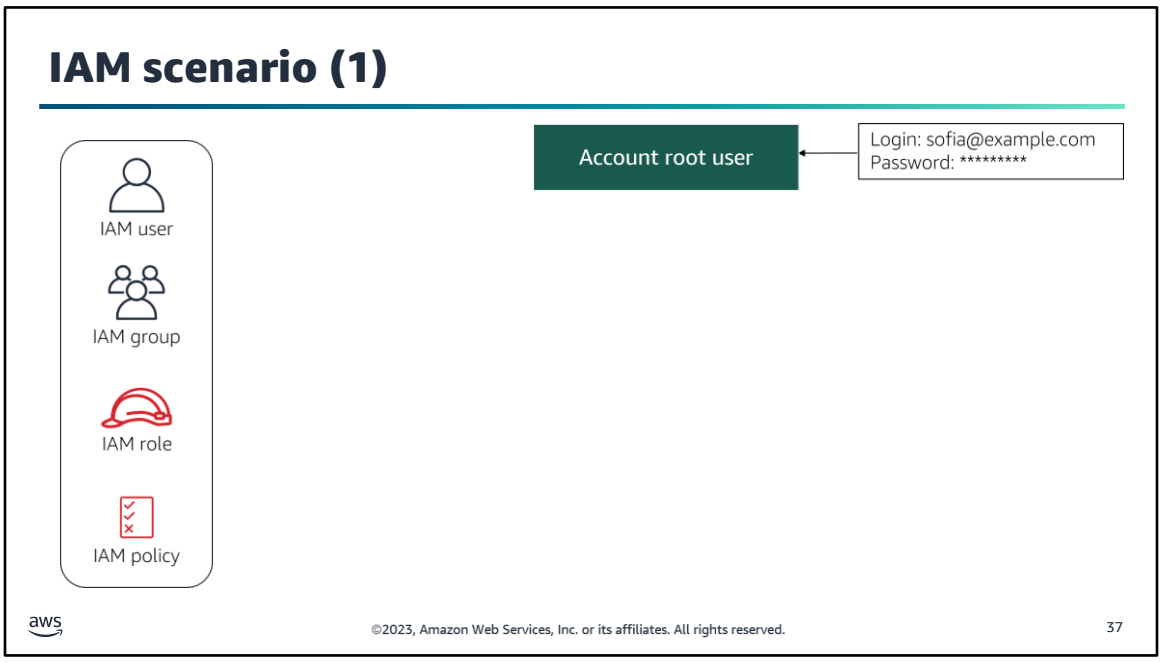
In its totality, this IAM policy allows access to specific DynamoDB and S3 resources. It then explicitly denies access to any other DynamoDB or S3 resources in the account.



When you grant permissions to users, groups, roles, and resources, follow the standard security advice of **the principle of least privilege**. This practice means that you grant only the permissions that are needed to perform a task.

It's more secure to start with a minimum set of permissions and grant additional permissions as needed. It provides better security than starting with permissions that are too permissive and then trying to restrict them later. To define the correct set of permissions, you must do some research to determine what access is needed to accomplish a specific task.

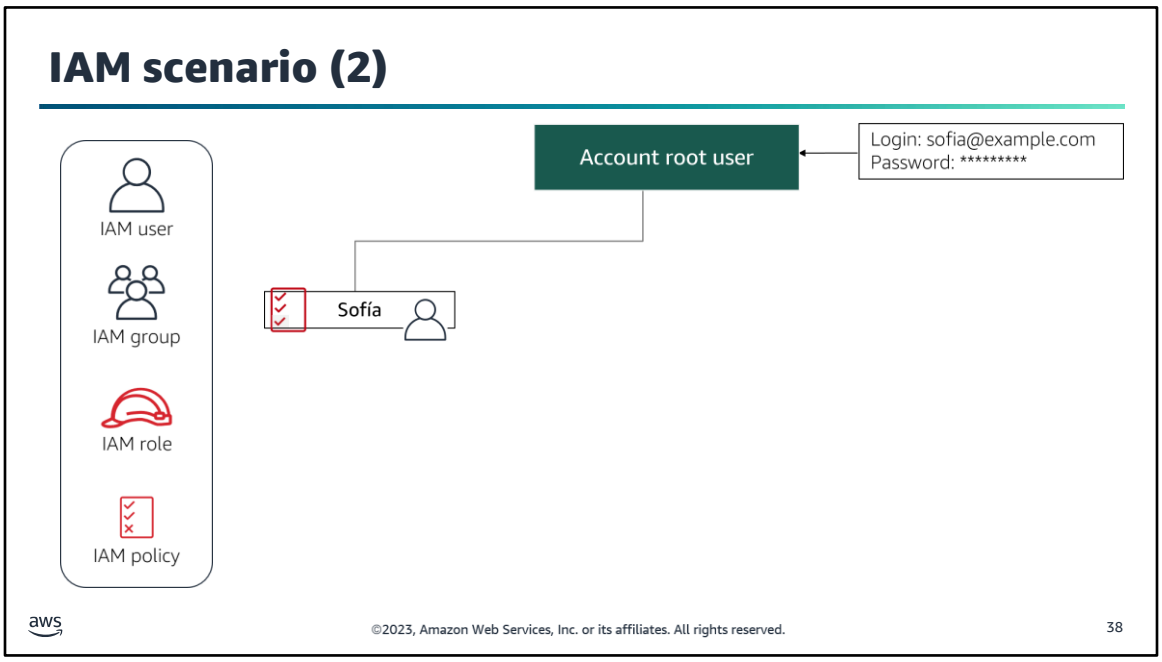
When you create IAM policies, determine what your users need to do, and then craft policies that allow them to perform only those tasks. Similarly, create policies for individual resources that identify precisely who is allowed to access the resource, and allow only the minimal permissions for those users. For example, perhaps developers should be allowed to create EC2 instances in production environments, but not to stop or terminate the instances.



Now that you learned about the principle of least privilege and how IAM policy logic works, consider the following scenario.

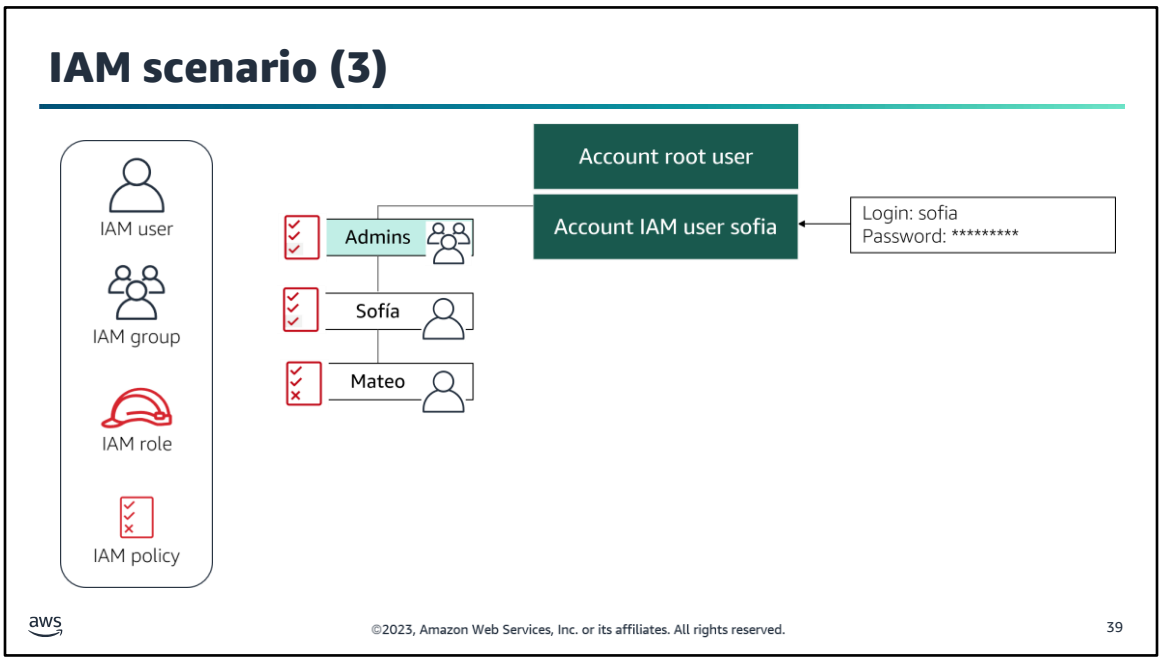
In this scenario, Sofía works for the café. She decides to use IAM features to configure access for everyone on her team who needs access to the café's AWS account. Sofía was the person who created the AWS account, so she has the account root user credentials. She logs in to the AWS Management Console with the email address that was used to create the account. As a result, she is logged in as the account root user.

The AWS account root user has full access to all resources in the account. These resources include billing information, personal data in the user profile, and all resources that were created in any AWS services in the account. You can't control the privileges of the AWS account root user credentials.



AWS recommends that you not use root user credentials for day-to-day tasks. Instead, use the account root user to create one or more IAM users. Then, keep the root user credentials in a secure location. For most ongoing account access and management tasks, you can use IAM user credentials.

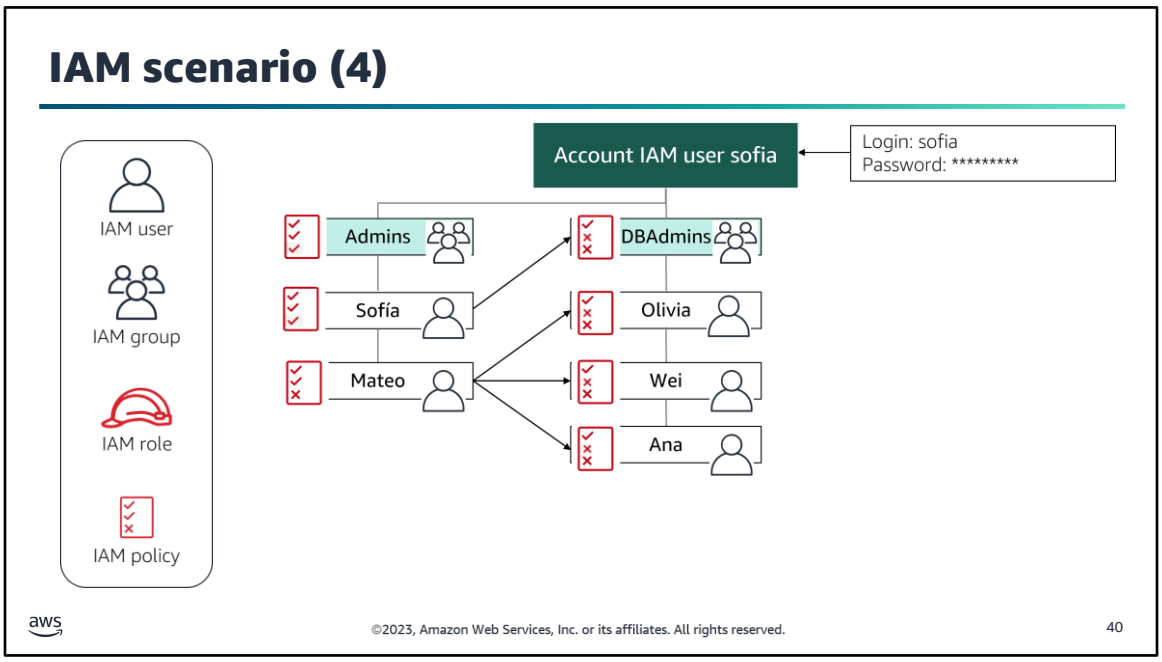
Following best practices, Sofía creates an IAM user for herself to use as her primary login user. Next, she applies a set of permissions to her IAM user. She gives her IAM user full access to all AWS services. She then logs out as the account root user and logs in as the IAM user.



Now that she is authenticated as the new IAM user, Sofía creates an IAM group for administrators, which is named *Admins*. She attaches one or more IAM policies that grant administrative access to different AWS services to this IAM group. She then adds her IAM user to this group. Because she is a member of the IAM group, her IAM user inherits the permissions that are attached to the group.

As the *Sofía* user, she defines another IAM user, *Mateo*, who is one of the AWS consultants she hired to help her manage the account. She wants Mateo to help administer the account, but she wants to reserve the ability to manage IAM group policies for herself. Therefore, she attaches a custom IAM policy to the *Mateo* IAM user. In this custom policy, she *explicitly denies* him access to creating IAM groups and attaching policies to them.

Sofía then adds the *Mateo* IAM user to the *Admin* IAM group, which grants Mateo all of the same administrative permissions as Sofía. However, his IAM user policy includes an explicit denial of IAM group permissions. Consequently, that part of the group policy is overridden for him, and he is denied those specified permissions.

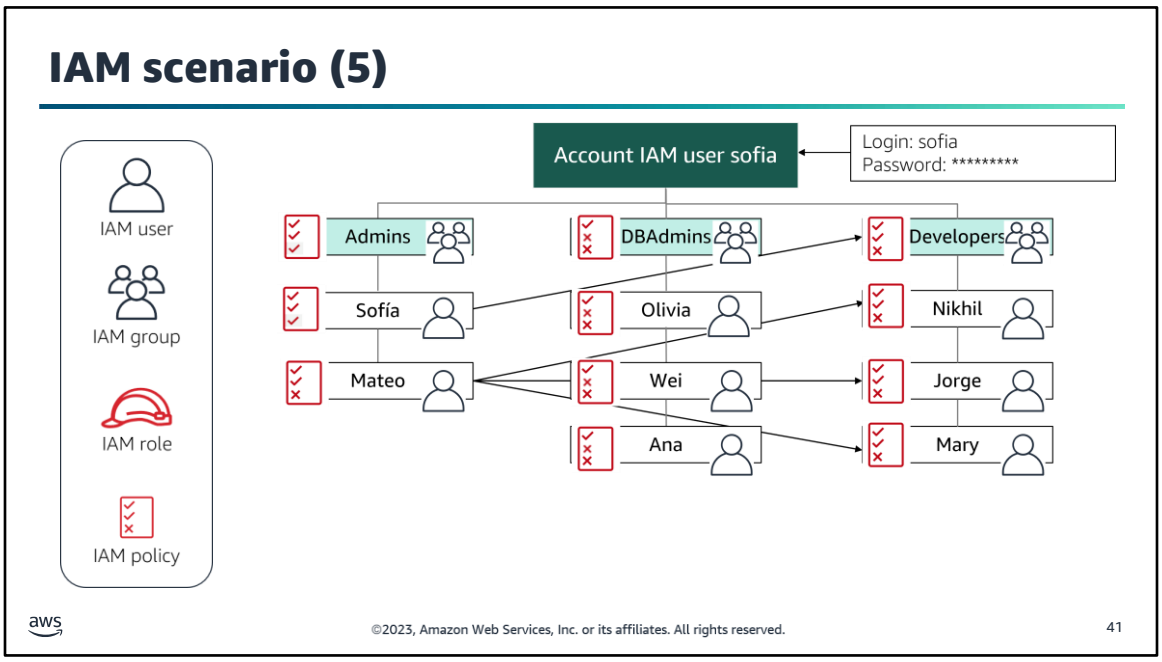


Next, Sofía creates a new IAM group for her company's database administrators.

Applying the *principle of least privilege*, the permissions that she attaches to the *DBAdmins* group are more restrictive than the permissions for the *Admins* group. She applies these more rigorous restrictions because the database administrators don't need Administrator permissions across all AWS services to do their work. For example, she grants them full access to DynamoDB (a database service that you will learn more about later in this course). However, she does not grant them full access to Amazon EC2.

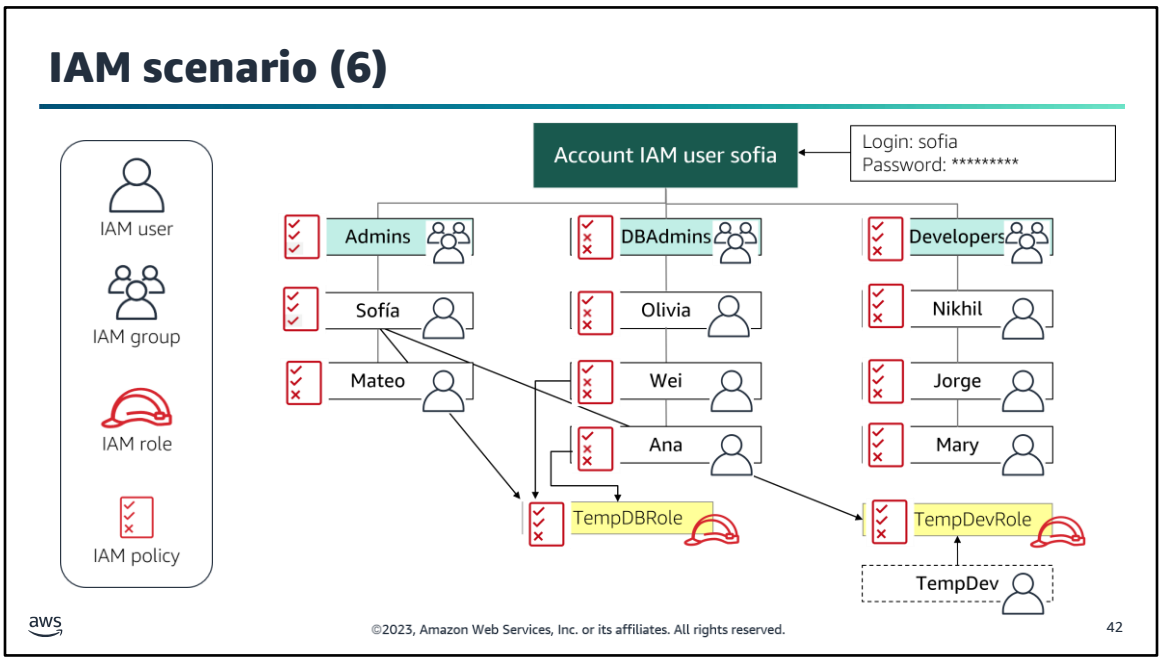
Sofía then asks Mateo to create the IAM users for the people they want to grant database administrator permissions to. He does that task, and also places those users into the *DBAdmins* IAM group.

The users in the *DBAdmins* group inherit the access permissions that are specified in the policies of the group because they are members of it. Although Mateo can't create IAM groups or manage their policies, he still has permissions to add users to the *DBAdmins* group and remove them from the *DBAdmins* group. (These permissions come from the policy that's attached to the *Admins* group, which he belongs to.)



Sofía and Mateo repeat the process again. This time, they create a **Developers** IAM group, create IAM users for the developers, and add the users to the IAM group.

In some cases, the policies that are attached to the *Developers* group might be too permissive for an individual user. The policies might grant more permissions than Sofía or Mateo want to grant to a specific user who's a member of the group. In such cases, they attach an IAM policy directly to the user, therefore denying this user the ability to take specific actions.

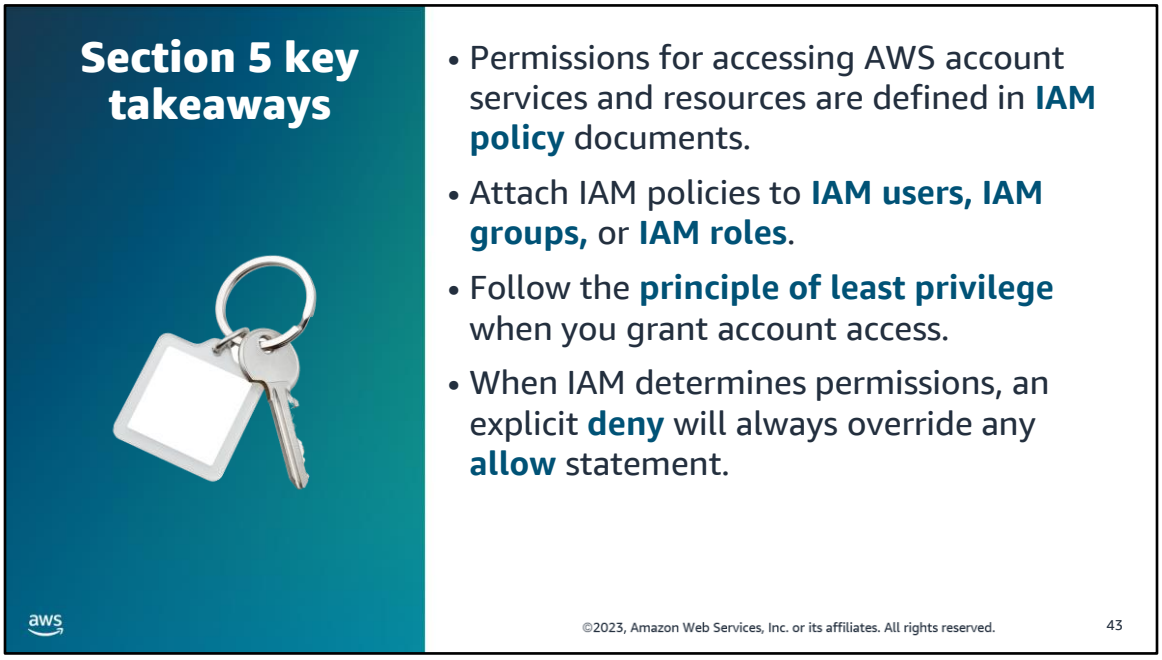


Sofía and Mateo have now created the basic permissions structure that Sofía wanted. However, a few days later, Sofía decides that she wants to hire someone on a temporary contract basis, to work on a specific project. That person will need some access to develop an application that uses AWS services. Sofía also wants to temporarily grant additional database permissions to two members of the DBAdmins group to work on a project that uses Amazon RDS. However, she doesn't want to grant that additional access to all members of the *DBAdmins* group, who only need access to Amazon DynamoDB.

In this scenario, the **DBAdmins IAM group** grants full access to the Amazon DynamoDB service, but doesn’t grant full access to the Amazon RDS service. A new project will require Wei and Ana to have access to Amazon RDS. Sofía decides that she can use IAM roles to grant the needed access.

First, Sofía creates a **TempDBRole** IAM role. She then attaches an IAM policy to that role that grants full Amazon RDS access. Finally, she temporarily attaches the role to two of the IAM users who are also part of the DBAdmins group, Wei and Ana. After the project is completed, she can remove them from membership in the role.

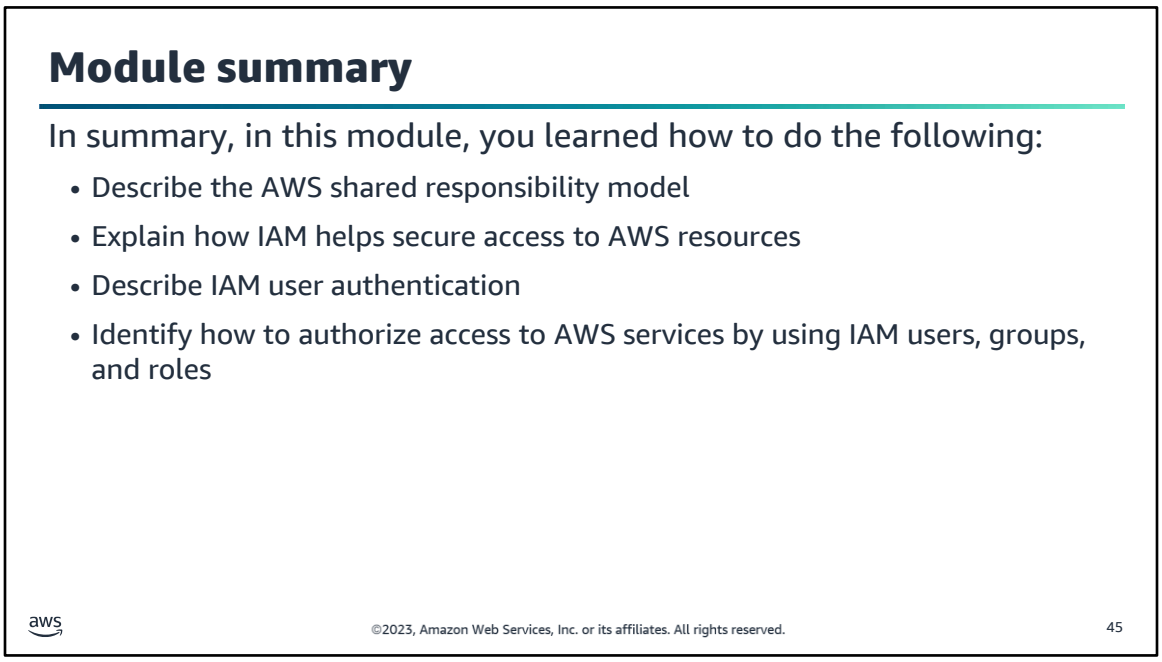
Sofía also creates a **TempDevRole** role. This role grants a more restrictive set of policies to a temporary staff member who was brought in to help work on a specific application. She defines a new IAM user that is named *TempDev*, but does not add the new user to the *Developers* group. Instead, she attaches the *TempDevRole* to the *TempDev* user. This way, temporary developers aren't granted the same broad account access as the permanent staff in the *Developers* IAM group. By defining this role, Sofía is also able to grant Ana, a *DBAdmins* group member, some temporary developer access so she can help with the project.



The following are the key takeaways from this section of the module:

- Permissions for accessing AWS account services and resources are defined in IAM policy documents.
- Attach IAM policies to IAM users, IAM groups, or IAM roles.
- Follow the principle of least privilege when you grant account access.
- When IAM determines permissions, an explicit deny will always override any allow statement.

It’s now time to review the module and wrap up with a knowledge check and discussion of a practice certification exam question.



In summary, in this module, you learned how to do the following:

- Describe the AWS shared responsibility model
- Explain how IAM helps secure access to AWS resources
- Describe IAM user authentication
- Identify how to authorize access to AWS services by using IAM users, groups, and roles

Complete the knowledge check

It is now time to complete the knowledge check for this module.

Sample exam question

A company is storing an access key (access key ID and secret access key) in a text file on a custom Amazon Machine Image (AMI). The company uses the access key to access DynamoDB tables from instances that are created from the AMI. The security team has mandated a more secure solution.

Which solution will meet the security team's mandate?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **Storing an access key**
- **On a custom Amazon Machine Image (AMI)**
- **To access DynamoDB tables**
- **More secure solution**

 47

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Responses

A company is **storing an access key** (access key ID and secret access key) in a text file **on a custom Amazon Machine Image (AMI)**. The company uses the access key **to access DynamoDB tables** from instances that are created from the AMI. The security team has mandated a **more secure solution**.

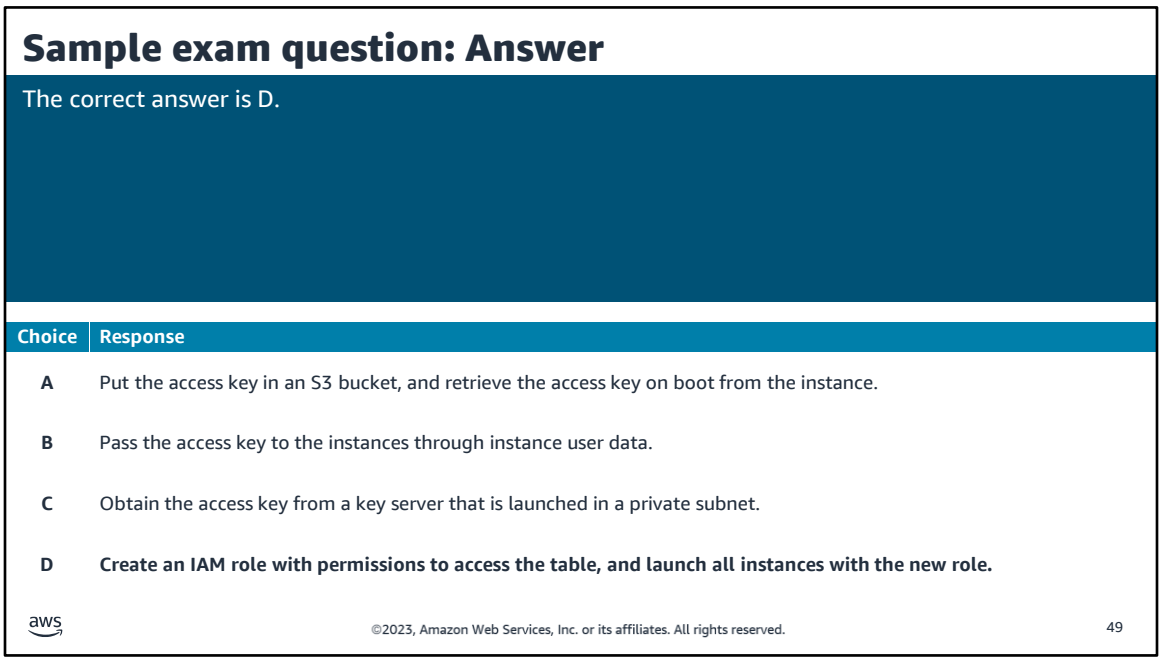
Which solution will meet the security team's mandate?

Choice	Response
A	Put the access key in an S3 bucket, and retrieve the access key on boot from the instance.
B	Pass the access key to the instances through instance user data.
C	Obtain the access key from a key server that is launched in a private subnet.
D	Create an IAM role with permissions to access the table, and launch all instances with the new role.

aws

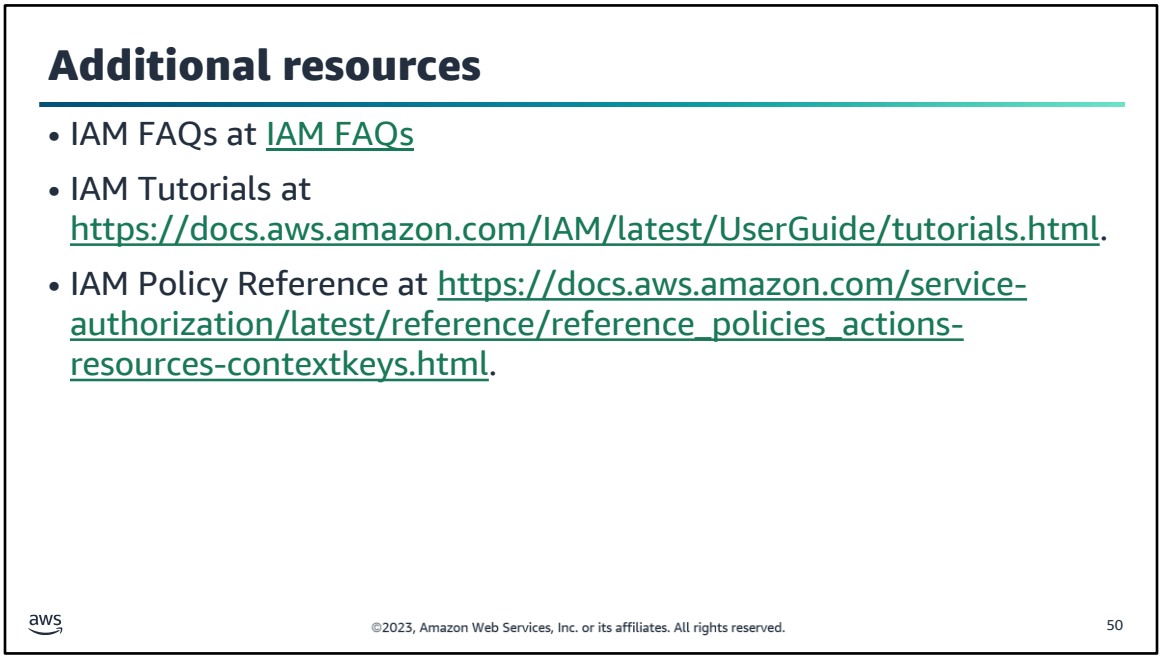
48

Now that we have bolded the keywords in this scenario, let us look at the answers.



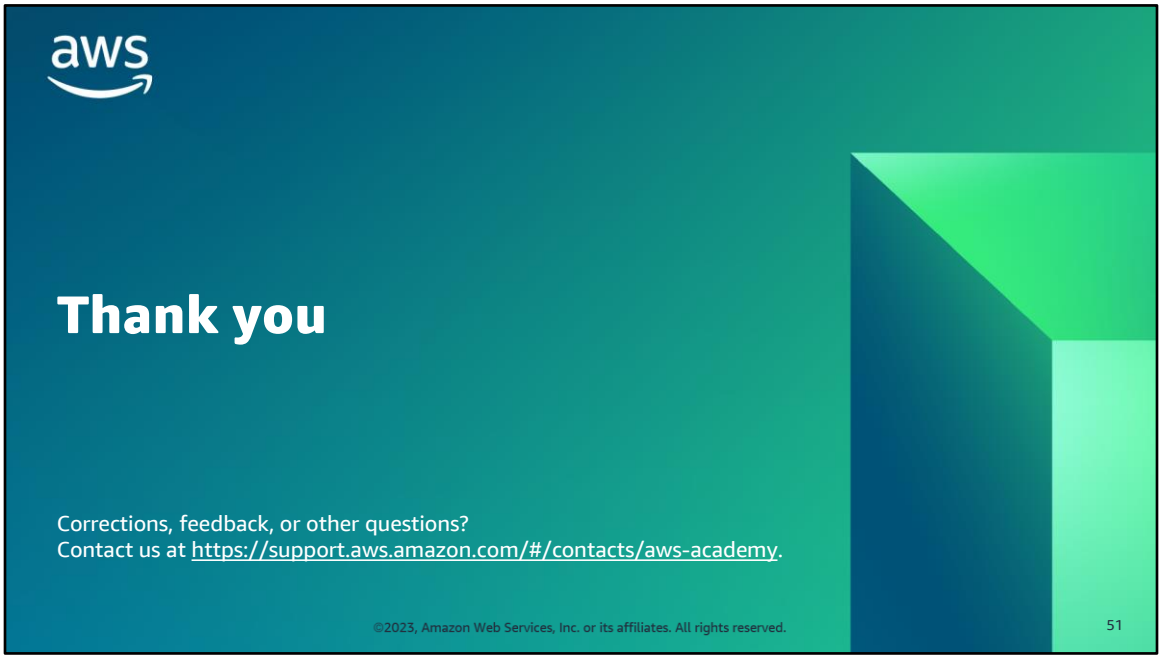
The correct answer is D. *Create an IAM role with permissions to access the table, and launch all instances with the new role.*

IAM roles for EC2 instances allow applications that run on the instance to access AWS resources without needing to create and store any access keys. Any solution that involves the creation of an access key also introduces the complexity of managing that secret.



For more information about the topics covered in this module, see:

- "AWS IAM FAQs" at https://aws.amazon.com/iam/faqs/.
- "IAM Tutorials" at https://docs.aws.amazon.com/IAM/latest/UserGuide/tutorials.html.
- "IAM Policy Reference" at https://docs.aws.amazon.com/service-authorization/latest/reference/reference_policies_actions-resources-contextkeys.html.



Welcome to Module 4: Securing Access to Cloud Resources.

